

Mikroelektronische Schaltungen und Systeme

Vorlesung 15 Digitale Bausteine

Prof. Dr.-Ing. Dr. h. c. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



Mikroelektronische Schaltungen und Systeme



1. Addierer
2. Komparator
3. Arithmetik-/Logikeinheit
4. Shifter und Rotators
5. Multiplikation und Division
6. Sequentielle Bausteine



Addierer

1

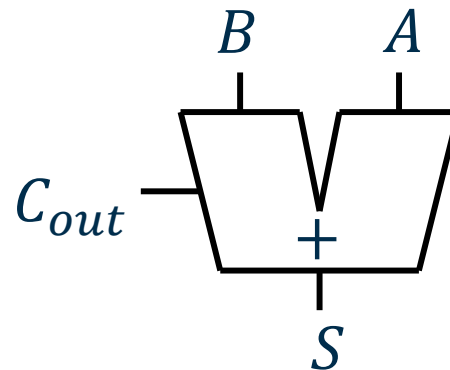
Addierer

1-Bit Half Adder

- Ein 1-Bit half adder erzeugt die Summe der beiden Eingänge A und B
 - Da der 1-Bit-Output S die Summe nicht speichern kann, wenn sowohl A als auch $B = 1$ sind, zeigt das carry out (Übertrag) Signal diesen Fall an.
- Es gelten folgende Gleichungen:

$$S = A \oplus B$$

$$C_{out} = A \wedge B$$



A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Referenz S. 240 [1]

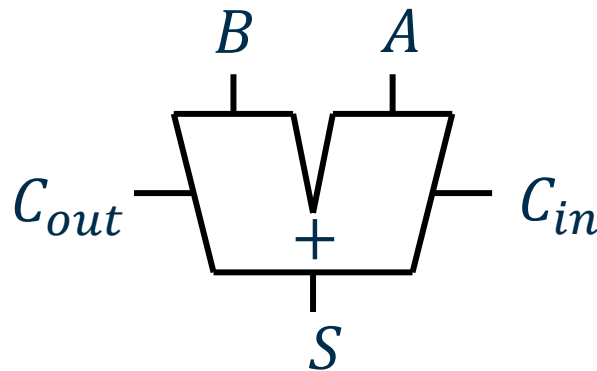
Addierer

Full Adder

- Der Full Adder erzeugt die Summe der drei Eingänge A , B , and C_{in} .
- Es gelten folgende Gleichungen:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \wedge B) \vee (A \wedge C_{in}) \vee (B \wedge C_{in})$$



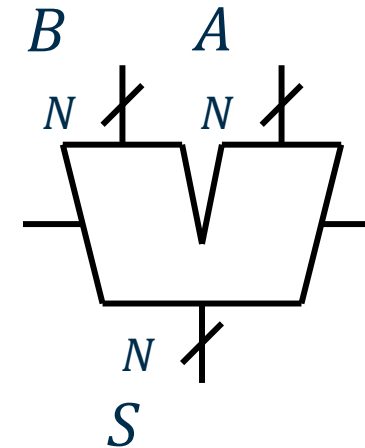
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Referenz S. 240 [1]

Addierer und Subtrahierer

Carry Propagate Adder

- Der **carry propagate adder (CPA)** addiert zwei N-Bit-Eingänge A und B sowie einen carry in C_{in} , um einen N-Bit-Output S und einen carry out C_{out} zu berechnen.
- Er überträgt den carry out in den carry in der nächsten Spalte.
- Gängige Implementierungen sind:
 - **Ripple-Carry Adder**
 - **Carry-lookahead Adder**
 - **Prefix Adder**

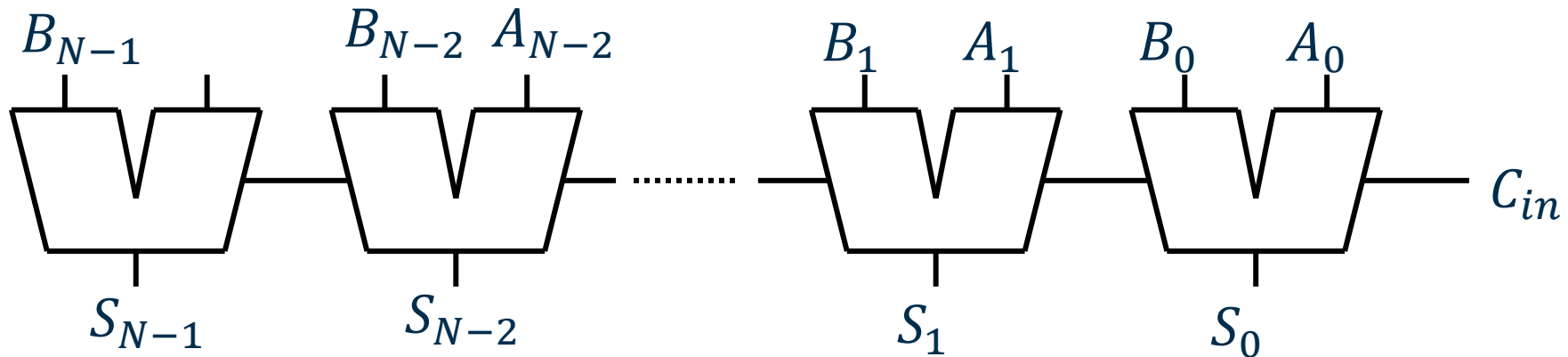


Referenz S. 240 [1]

Addierer

Ripple-Carry Adder

- Der ripple-carry adder ist eine Kette aus N Full adders.
- Die Ausbreitungsverzögerung steigt linear mit N .
 - $t_{pd_N_ripple_carry_adder} = N * t_{pd_full_adder}$



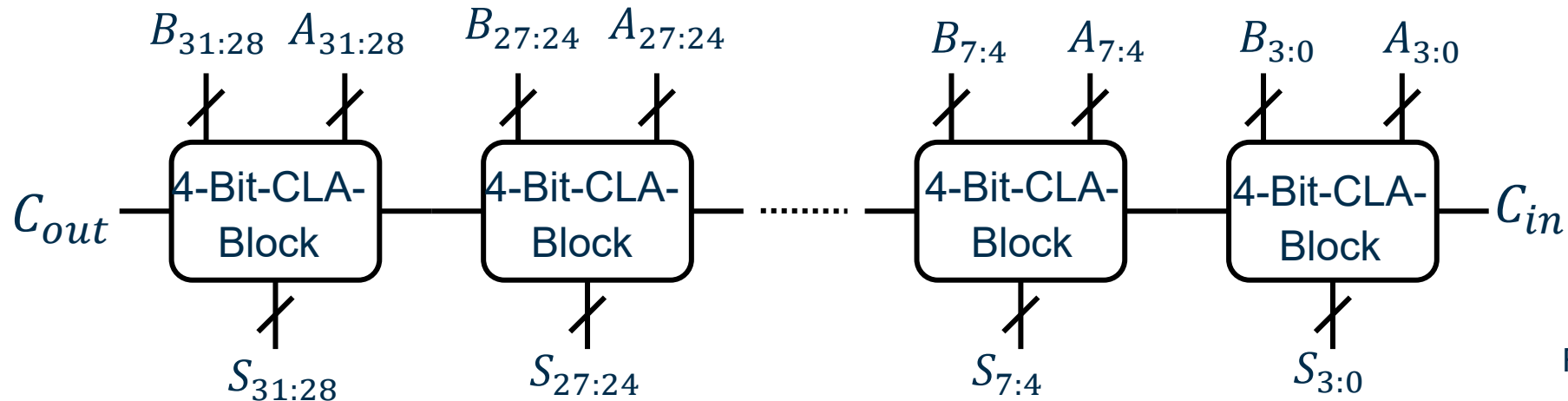
Referenz S. 240 [1]

Addierer

Carry Lookahead Adder Einführung

- Der **Carry-Lookahead Adder** (CLA) nutzt die Tatsache, dass der letzte carry out mit einer speziellen Schaltung (**Generate-and-Propagate-Logik**) für N-Bit-Eingaben schneller berechnet werden kann, als mit einer Kette von Volladdierern.
- Der Addierer ist in **CLA-Blöcke** unterteilt, die mit der Berechnung eines Teils der Summe beginnen, sobald die vorherige Schaltung den erforderlichen carry in liefert.

Beispiel 32-Bit-CLA mit 4-Bit-Blöcken



Referenz S. 241 [1]

Addierer

Carry Lookahead Adder

- Für bestimmte Konfigurationen der Eingänge A und B gilt für den Output $C_{out} = 1$.

➤ Für den Fall

$$G = A \wedge B = 1$$

wird gesagt, dass der Addierer das Carry-bit erzeugt (generates).

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Referenz S. 241 [1]

Addierer

Carry Lookahead Adder

- Bei bestimmten Konfigurationen der Eingänge A und B bildet der Output C_{out} den Input C_{in} ab.

➤ Für den Fall

$$P = A \oplus B = 1$$

wird gesagt, dass der Addierer das carry out bit **propagiert (propagates)**.

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Referenz S. 241 [1]

Addierer

Carry Lookahead Adder

- Zusammengefasst wird die Ausgabe C_{out} entweder durch **Propagation** oder **Generation** festgelegt.

➤ In Gleichungsform

$$C_{out} = (A \wedge B) \vee ((A \vee B) \wedge C_{in})$$

$$C_{out} = G \vee (P \wedge C_{in})$$

Anmerkung

- $P = A \vee B$ kann in Kombination für das Generate Signal verwendet werden.

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Referenz S. 241 [1]

Addierer

Carry Lookahead Adder

- Die Definitionen für **propagation** und **generation** können in erweiterter Form für **multiple-bit adder blocks** angewendet werden.

Definition

- $G_{i:j}$ ist das **generate** signal für einen Block aus den Spalten j bis i . Der Block erzeugt C_i , wenn die höchste Spalte generates oder eine andere Spalte generates und bis zum Ende des Blocks propagiert wird.
- $P_{i:j}$ ist das **propagate** signal für einen Block aus den Spalten j bis i . Der Block propagiert C_{j-1} zu C_i , wenn alle Spalten propagieren.

Beispiel

$$G_{3:0} = G_3 \vee P_3(G_2 \vee P_2(G_1 \vee P_1 G_0))$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

Referenz S. 241 [1]

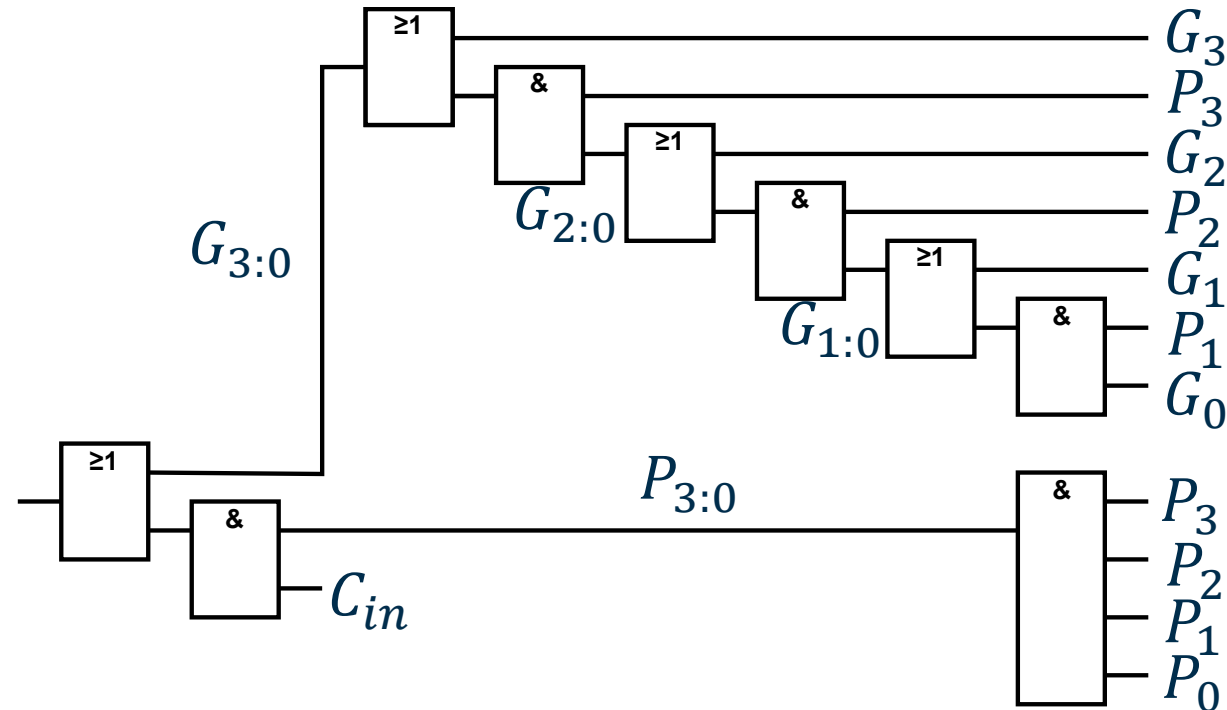
Addierer

Carry Lookahead Adder

- Der carry out C_i eines Blocks kann dann wie folgt berechnet werden:

$$C_i = G_{i:j} \vee P_{i:j} C_{j-1}$$

Beispiel für einen 4-Bit-CLA-Block

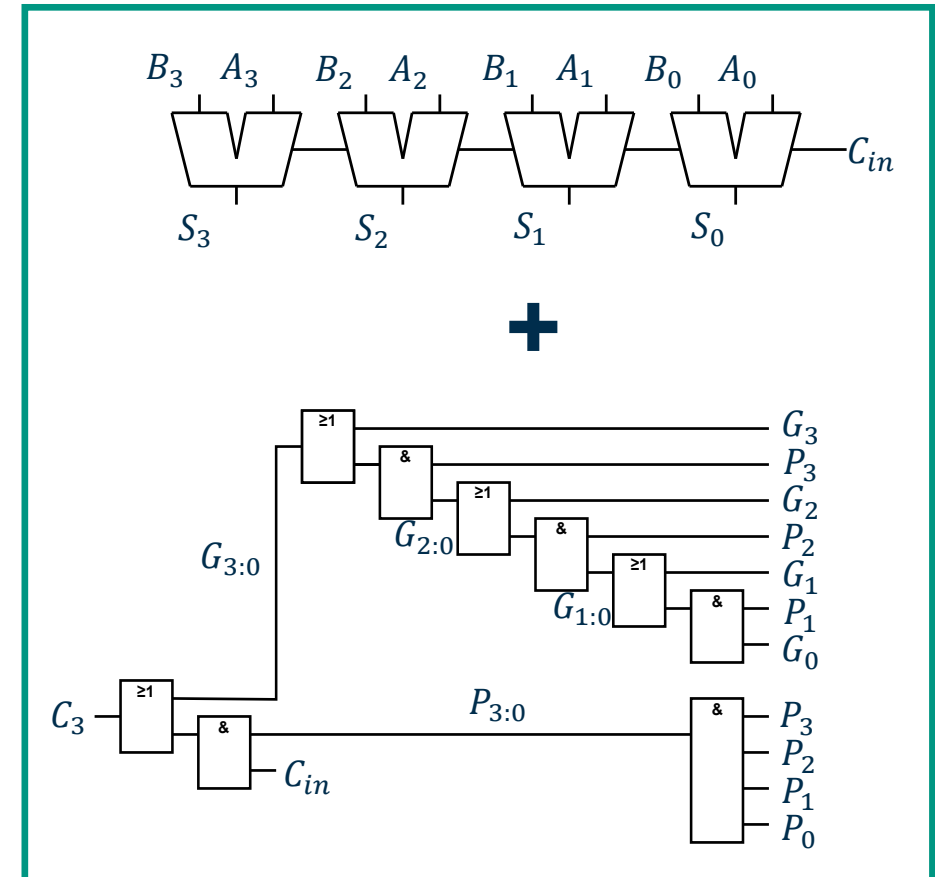
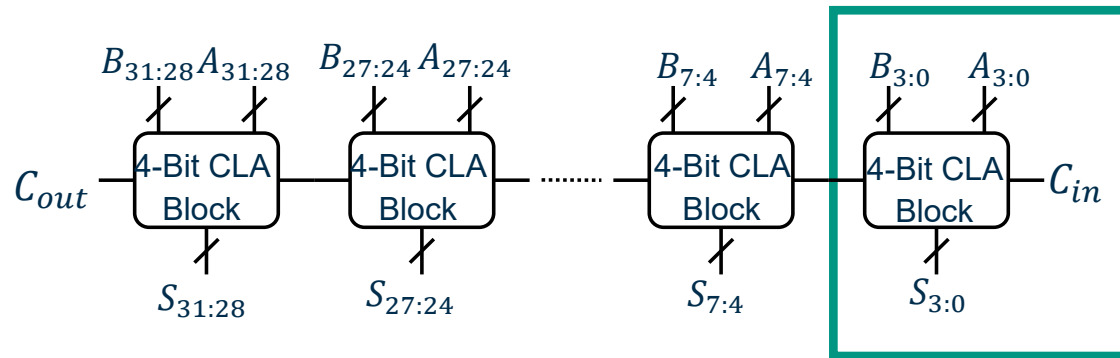


Referenz S. 242 [1]

Addierer

Carry Lookahead Adder

Beispiel 32-Bit-Carry-Lookahead-Addierer (CLA)



Referenz S. 242 [1]

Addierer

Prefix Adder

- Der Prefix Adder erweitert die generate und propagate Logik des Carry-Lookahead-Addierers, um das generate- und propagate Signal von Spalte “-1” zu jeder Spalte i zu berechnen.

Anmerkung

Spalte “-1” ist ein Konstrukt zur Vereinfachung der Abhängigkeiten, da $G_{-1} = C_{in}$ und $P_{-1} = 0$.

- Daher wird kein carry in von Spalte 0 durch die Blöcke propagiert ($P_{-1:-1}, P_{0:-1}, P_{1:-1}, \dots, P_{N-2:-1}, P_{N-1:-1} = 0$), sondern in Spalte “-1” generiert.
 - $C_{i-1} = G_{i-1:-1} \vee P_{i-1:-1} C_{-2} = G_{i-1:-1}$
 - $S_i = (A_i \oplus B_i) \oplus C_{i-1} = (A_i \oplus B_i) \oplus G_{i-1:-1}$

Referenz S. 243 [1]

Addierer

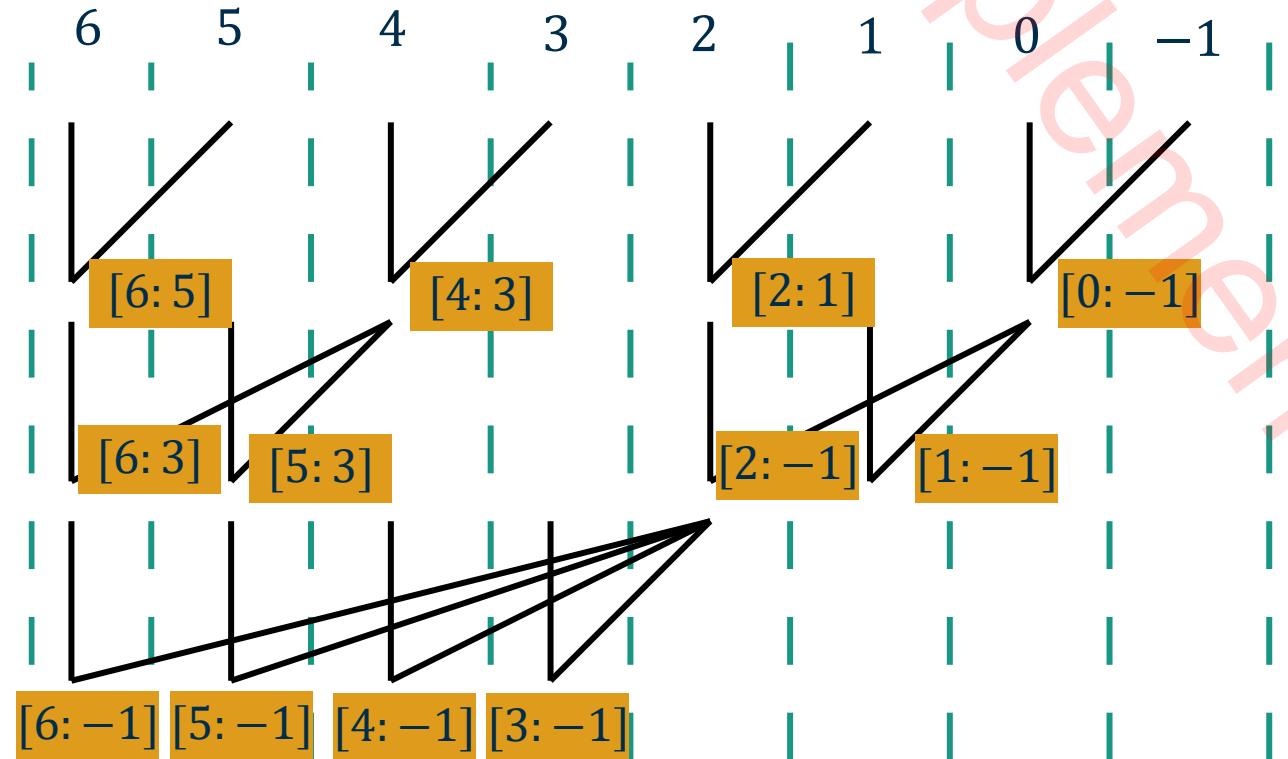
Präfixaddierer

- Es gelten die folgenden Gleichungen:

- $G_{i:j} = G_{i:k} \vee P_{i:k} G_{k-1:j}$
- $P_{i:j} = P_{i:k} P_{k-1:j}$

zur Berechnung der Blocksignale aus **zwei** Subblöcken.

- Hierarchisch** berechnet der prefix adder $G_{i:j}$ und $P_{i:j}$ für zwei aufeinanderfolgende Unterblöcke $[i:k]$ und $[k-1:j]$, wobei die ersten Subblöcke die Spalten selbst sind.

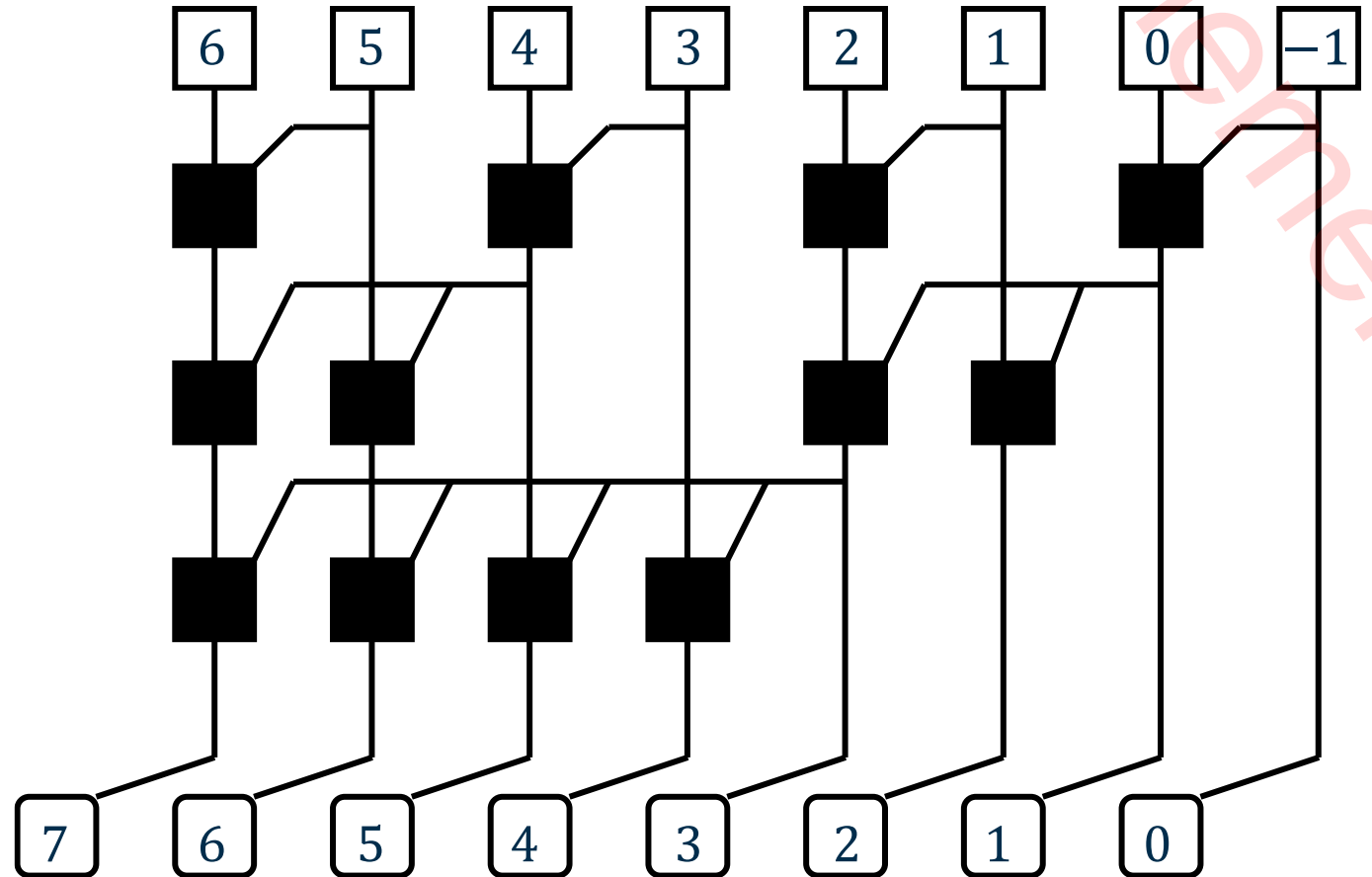
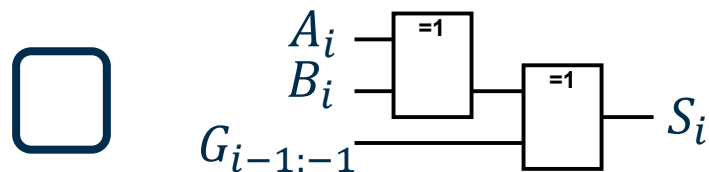
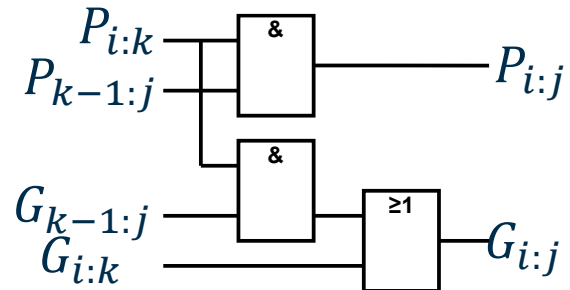
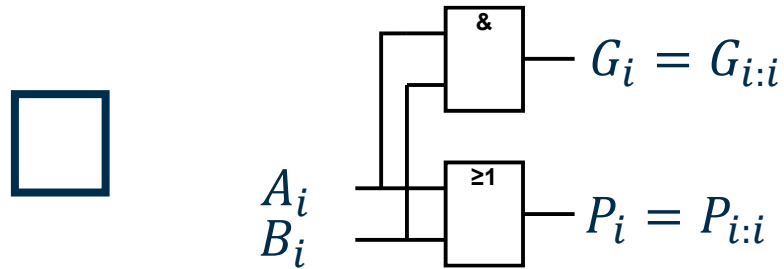


Referenz S. 244 [1]

Addierer

Präfixaddierer

Beispiel 8-bit prefix adder

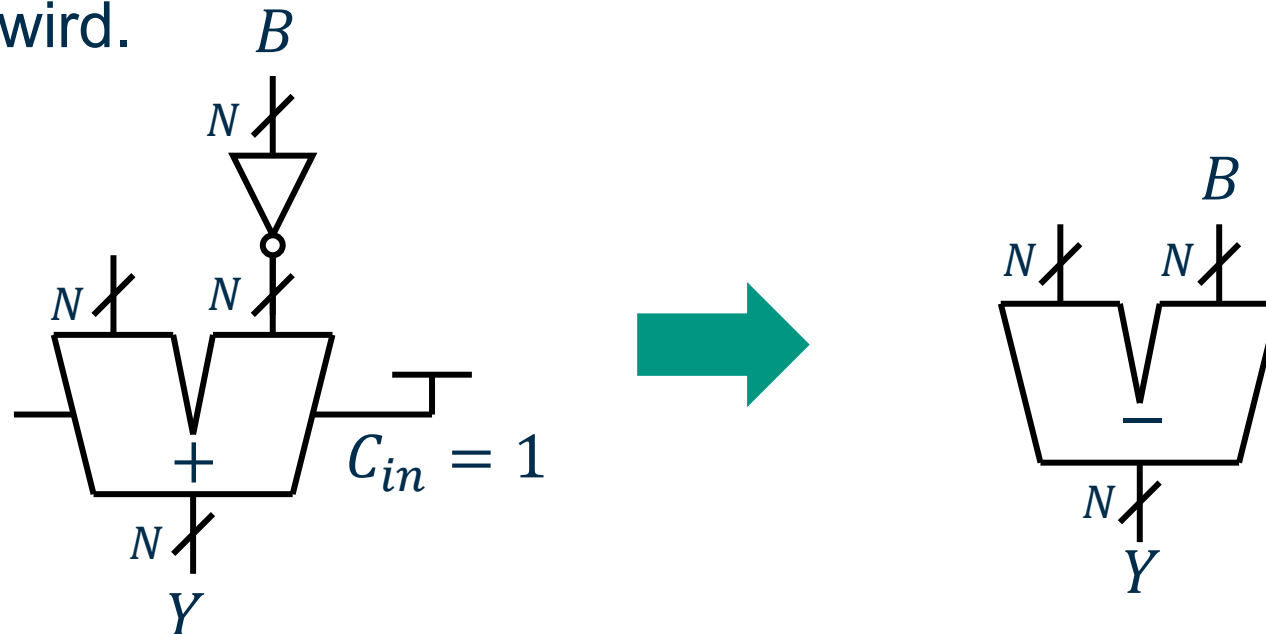


Referenz S. 244 [1]

Addierer

Subtraktion (Carry Propagate Adder)

- Die Subtraktion kann durch Negieren des zweiten Inputs unter Verwendung der Zweierkomplement-Darstellung durchgeführt werden. Das Negieren erfolgt durch Invertieren der Bits und Addieren von 1.
- Dies kann mit einem **CLA** erreicht werden, indem die zweite Eingabe invertiert und $C_{in} = 1$ gesetzt wird.



Referenz S. 246 [1]



Komparator

2

Komparator

Einführung

- Ein Komparator ermittelt für zwei Eingänge A und B , ob:
 - $A == B$
 - $A > B$
 - $A < B$
- Gängige Arten von Komparatoren sind:
 - Equality Comparator
 - Magnitude Comparator

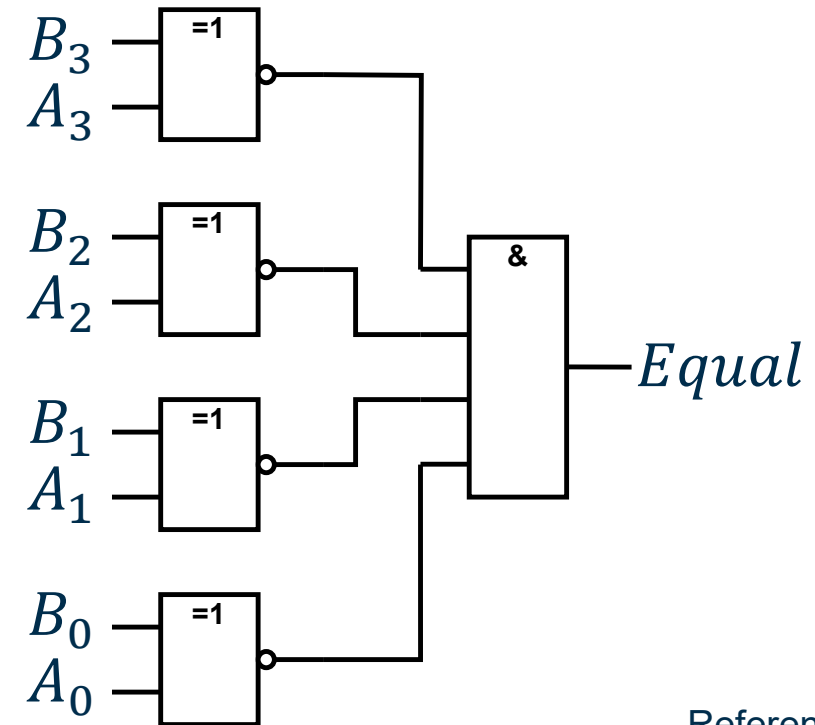
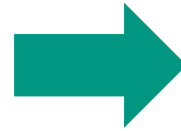
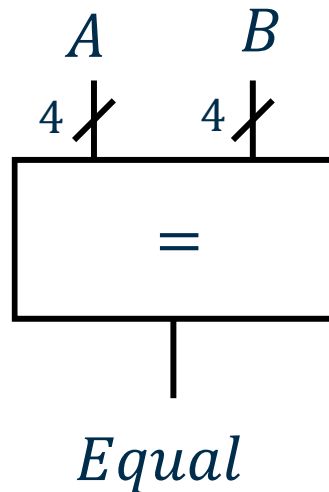
Referenz S. 246 [1]

Komparator

Equality Comparator

- Der Equality Comparator ermittelt für zwei Eingänge A und B ob $A == B$

Beispiel 4-bit Equality Comparator

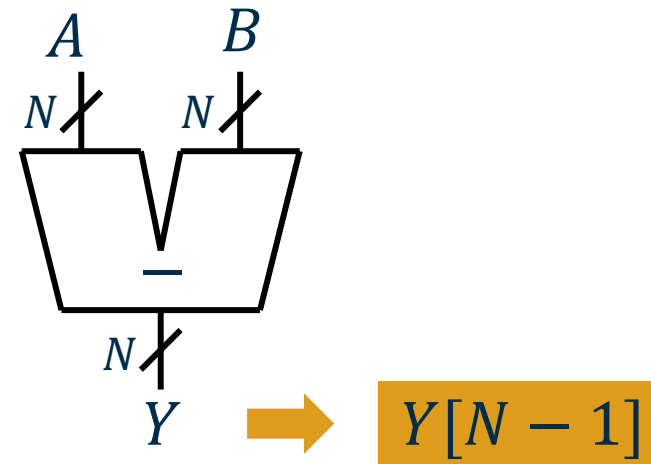


Referenz S. 247 [1]

Komparator

Magnitude Comparator

- Der Magnitude Comparator ermittelt für vorzeichenbehaftete (signed) Eingänge A und B ob:
 - $A \geq B$ oder $A < B$
- Dies geschieht durch Berechnung von $A - B$ und Überprüfung des **Most Significant Bit (MSB)** des vorzeichenbehafteten Ergebnisses. (a.k.a. Vorzeichenbit / sign bit)
 - $Y[N - 1] = 0 \Rightarrow A \geq B$
 - $Y[N - 1] = 1 \Rightarrow A < B$



Referenz S. 247 [1]

Komparator

Magnitude Comparator

Anmerkung

Der **Magnitude Comparator** funktioniert nicht bei **overflow** (das Ergebnis des Subtrahierers liegt nicht innerhalb des darstellbaren Bereichs der spezifizierten Anzahl von Bits).

In diesem Fall ist das sign bit nicht signifikant.

Beispiel Zweierkomplement in 5 Bit und 4 Bit

4 Bit		[-8:7]	5-Bit		[-16:17]	
	0111	$\Rightarrow 7$		00111	$\Rightarrow 7$	
-	1010	$\Rightarrow -6$	-	11010	$\Rightarrow -6$	
	1101	$\Rightarrow -3$		01101	$\Rightarrow 13$	Korrekt
		!Overflow				

Referenz S. 247 [1]

Komparator

Magnitudenkomparator

Anmerkung

- Bei vorzeichenbehafteten Ganzzahlen tritt bei der Subtraktion ein Overflow auf, wenn:
 - A und B haben unterschiedliche Vorzeichen und das Ergebnis hat ein anderes Vorzeichen als A .
 - $(\text{sign}(A) \neq \text{sign}(B)) \wedge (\text{sign}(A - B) \neq \text{sign}(A))$
- Um den Overflow zu erkennen, muss dies überprüft werden.



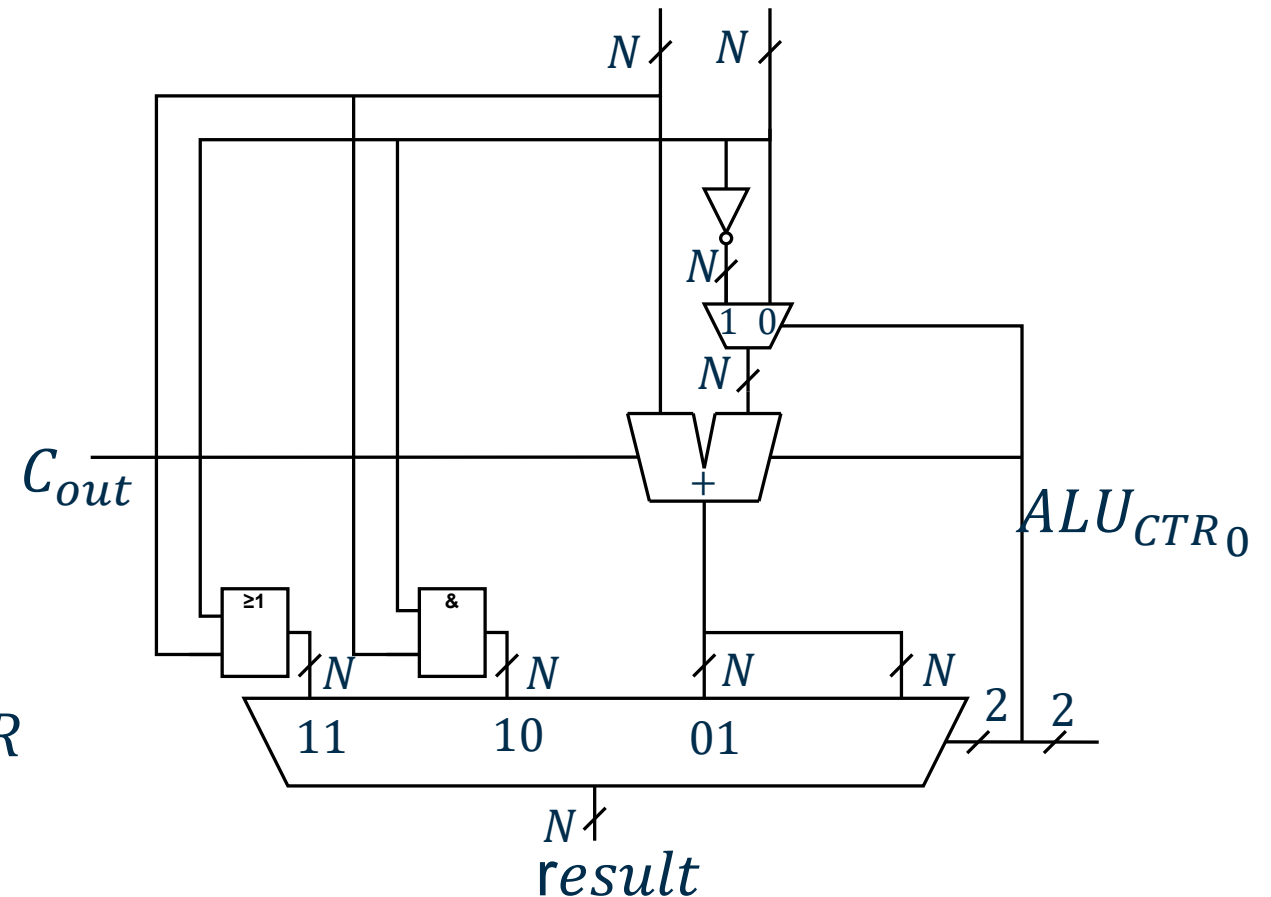
Arithmetic/Logic Unit

3

Arithmetic/Logic Unit

N-Bit Arithmetic/Logic Unit (ALU)

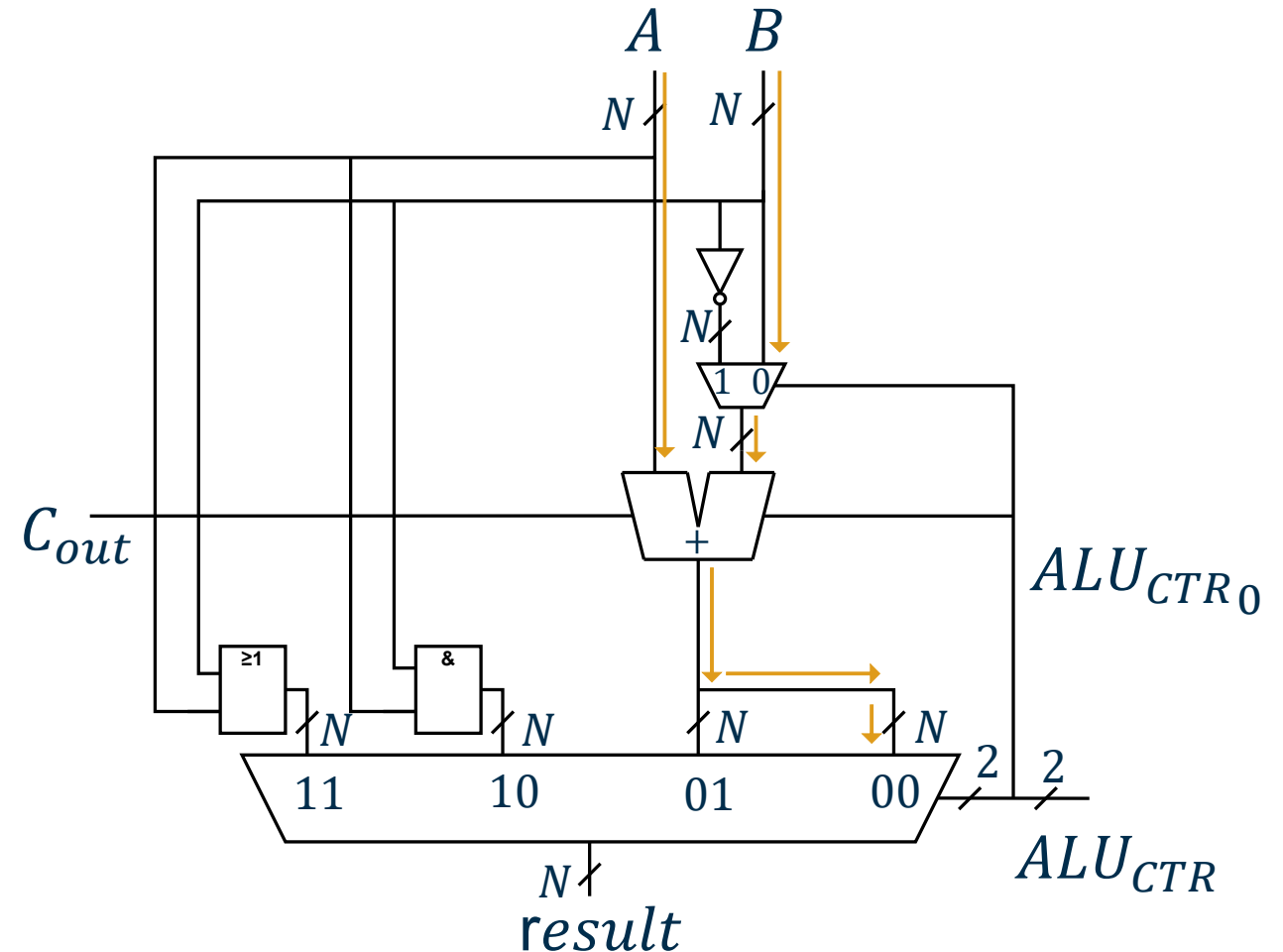
- Die **Arithmetic/Logic Unit (ALU)** kann verschiedene mathematische und logische Operationen kombinieren.
- Die vereinfachte ALU rechts kann folgende Operationen ausführen:
 - Addition
 - Subtraktion
 - N-Bit-AND-Operation
 - N-Bit-ODER-Operation
- Das **2-Bit-ALU control signal** ALU_CTR wählt den operation mode aus.



Arithmetic/Logic Unit

ALU Control Signal

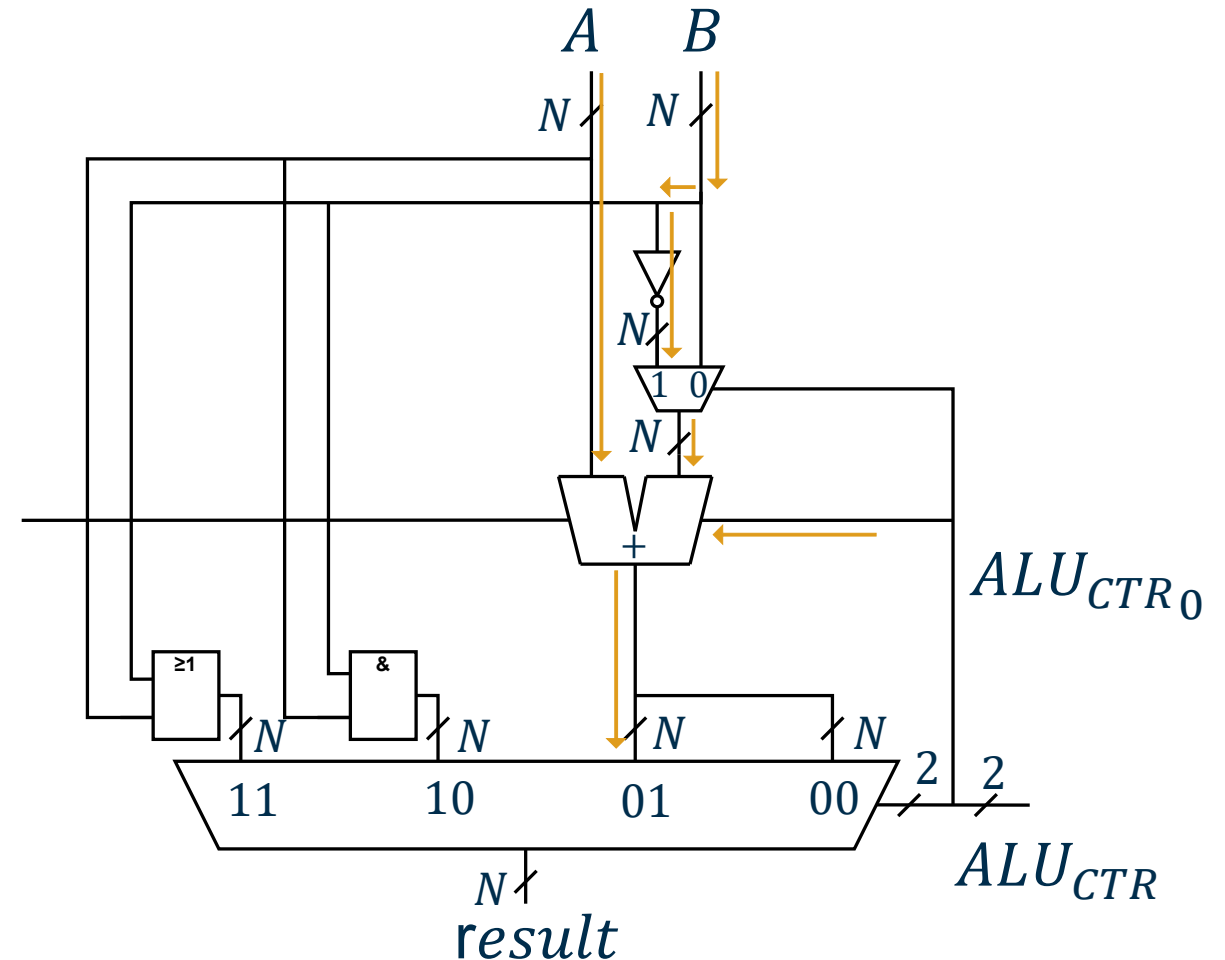
- $ALU_{CTR} = 00$
 - Addition $A + B$
- $ALU_{CTR} = 01$
 - Subtraction $A - B$
 - ALU_{CTR_0} wählt den invertierten B Input and setzt $C_{in} = 1$.
- $ALU_{CTR} = 10$
 - AND operation $A \wedge B$
- $ALU_{CTR} = 11$
 - OR operation $A \vee B$



Arithmetic/Logic Unit

ALU Control Signal

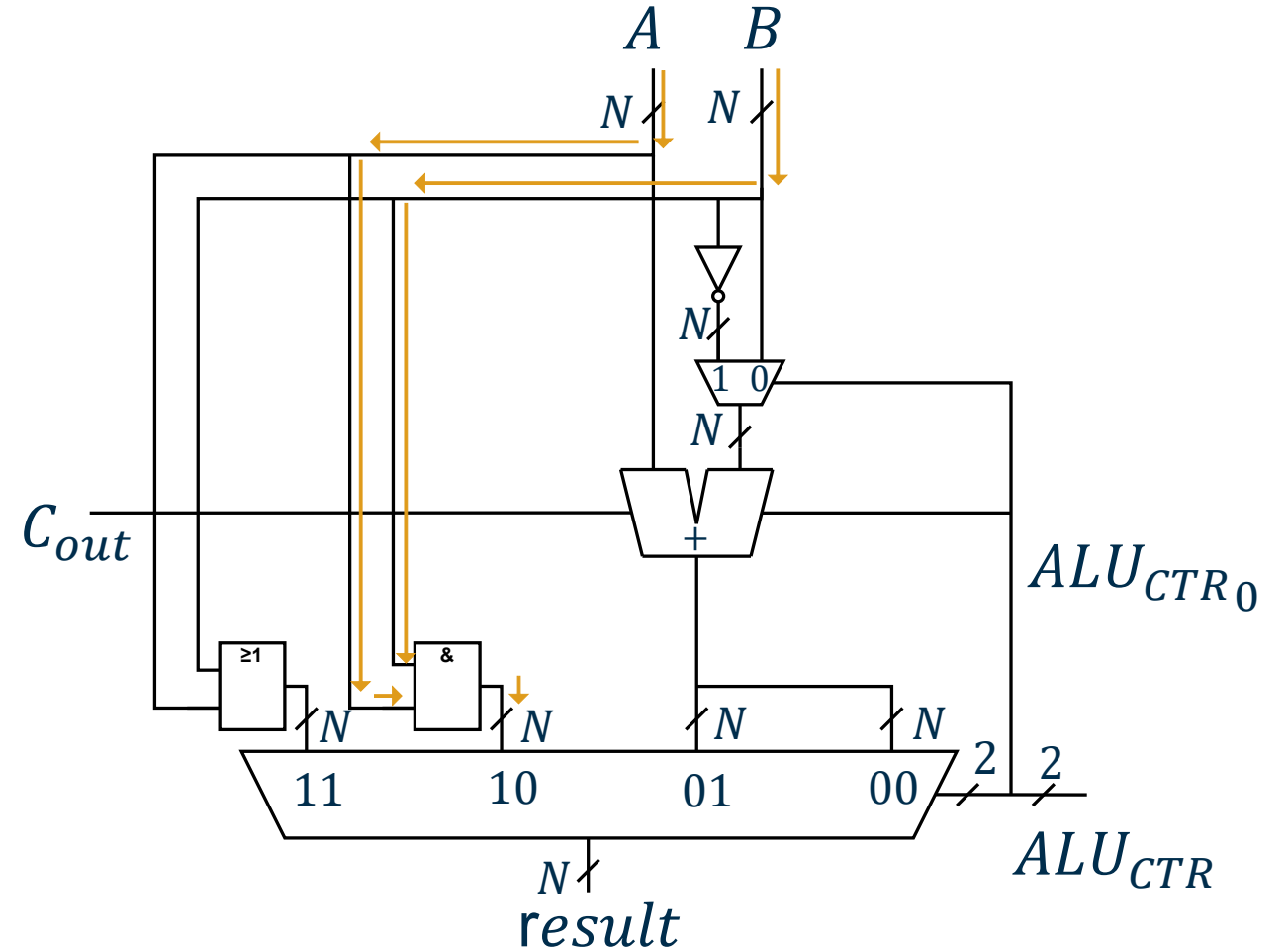
- $ALU_{CTR} = 00$
 - Addition $A + B$
- $ALU_{CTR} = 01$
 - Subtraction $A - B$
 - ALU_{CTR_0} wählt den invertierten B Input and setzt $C_{in} = 1$.
- $ALU_{CTR} = 10$
 - AND operation $A \wedge B$
- $ALU_{CTR} = 11$
 - OR operation $A \vee B$



Arithmetic/Logic Unit

ALU Control Signal

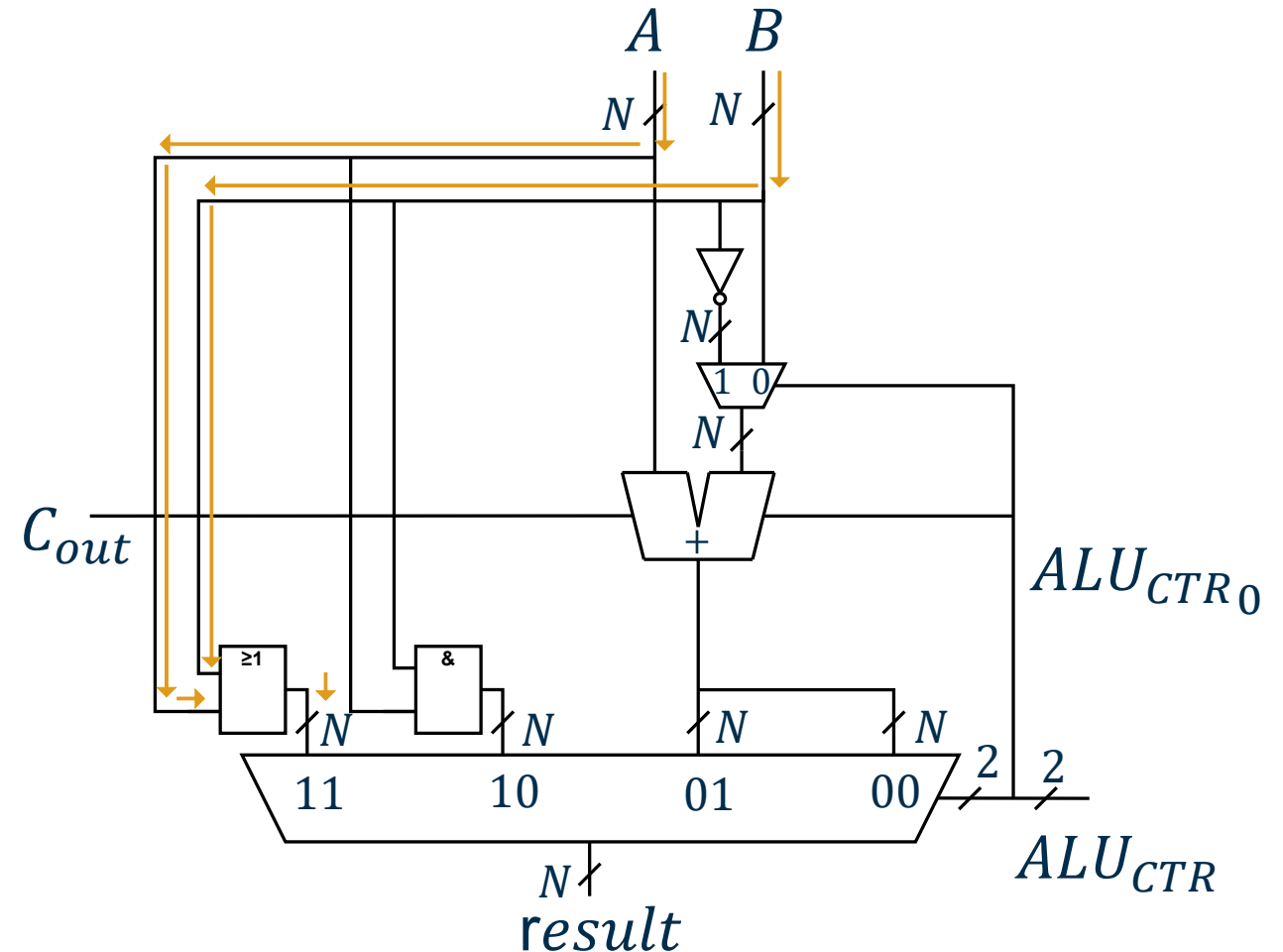
- $ALU_{CTR} = 00$
 - Addition $A + B$
- $ALU_{CTR} = 01$
 - Subtraction $A - B$
 - ALU_{CTR_0} wählt den invertierten B Input and setzt $C_{in} = 1$.
- $ALU_{CTR} = 10$
 - AND operation $A \wedge B$
- $ALU_{CTR} = 11$
 - OR operation $A \vee B$



Arithmetic/Logic Unit

ALU Control Signal

- $ALU_{CTR} = 00$
 - Addition $A + B$
- $ALU_{CTR} = 01$
 - Subtraction $A - B$
 - ALU_{CTR_0} wählt den invertierten B Input and setzt $C_{in} = 1$.
- $ALU_{CTR} = 10$
 - AND operation $A \wedge B$
- $ALU_{CTR} = 11$
 - OR operation $A \vee B$



Arithmetic/Logic Unit

Output Flags

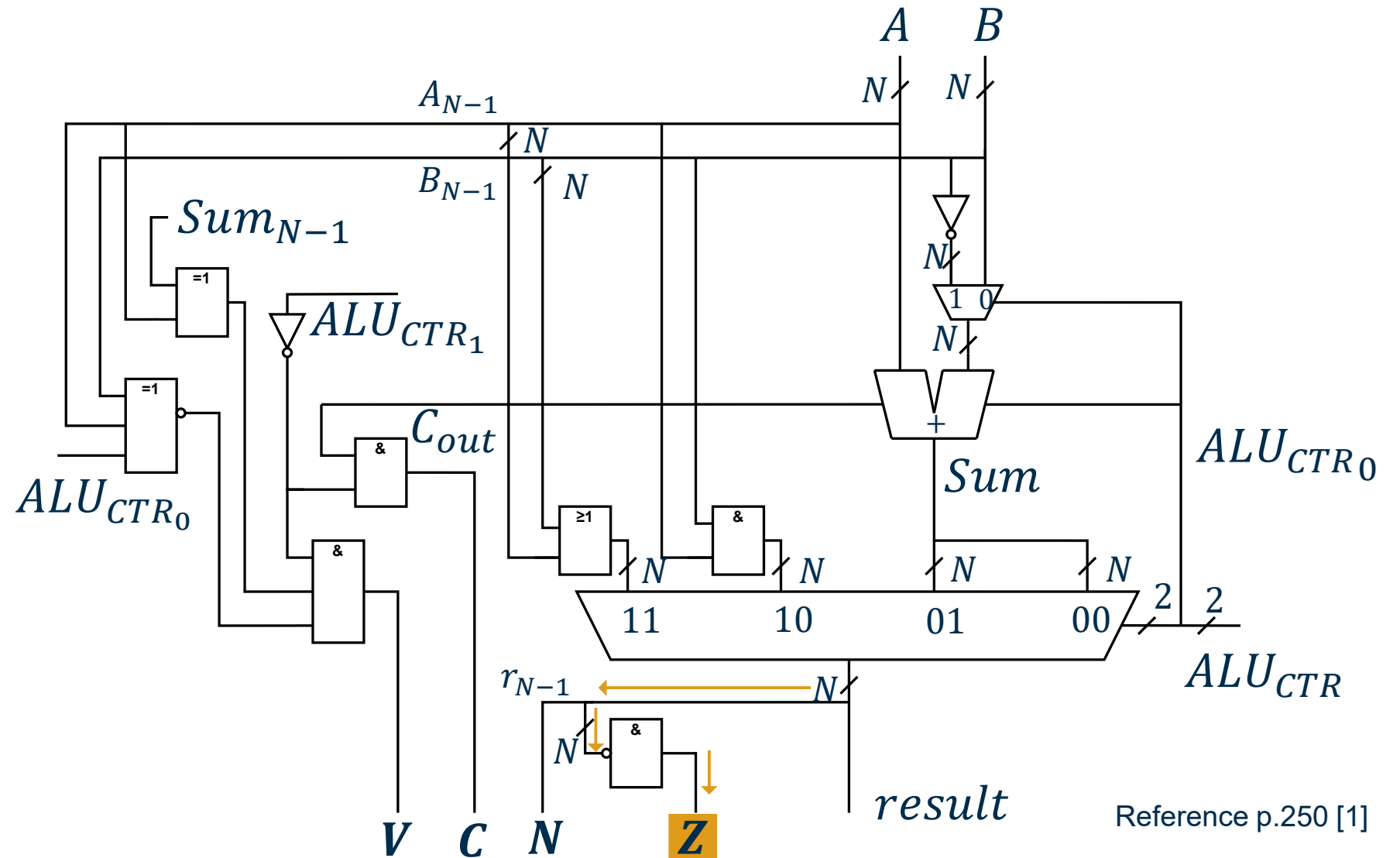
- In der Regel verfügen ALUs auch über **output flags**, um Informationen über die Ausgabe anzuzeigen. Die folgenden Flags sind üblich:
 - **Z**: Zero-Flag
 - Das Ergebnis ist Null.
 - **N**: Negative-Flag
 - Das Ergebnis ist negativ.
 - **C**: Carry-Flag
 - Carry out ist gesetzt (für ADD- und SUB- mode).
 - **V**: Overflow-Flag
 - Overflow erkannt (für ADD- und SUB-Modus).

Siehe S. 250 [1]

Arithmetic/Logic Unit

Output Flags

- **Z: Zero-Flag**
 - Alle N Bits von *result* sind Null.

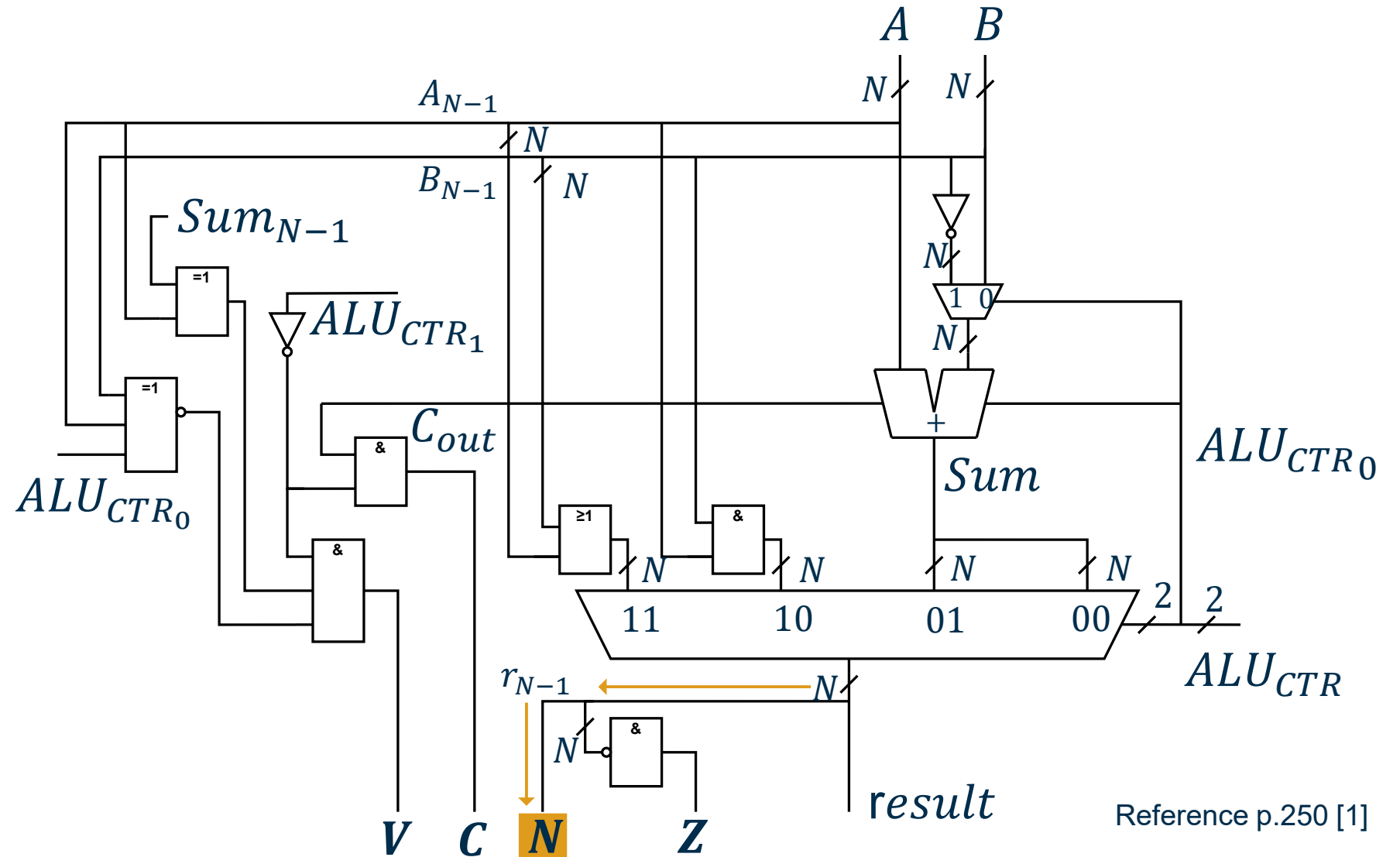


Reference p.250 [1]

Arithmetic/Logic Unit

Output Flags

- **N: Negative-Flag**
 - Das sign bit ($N - 1$) des Ergebnisses gesetzt.

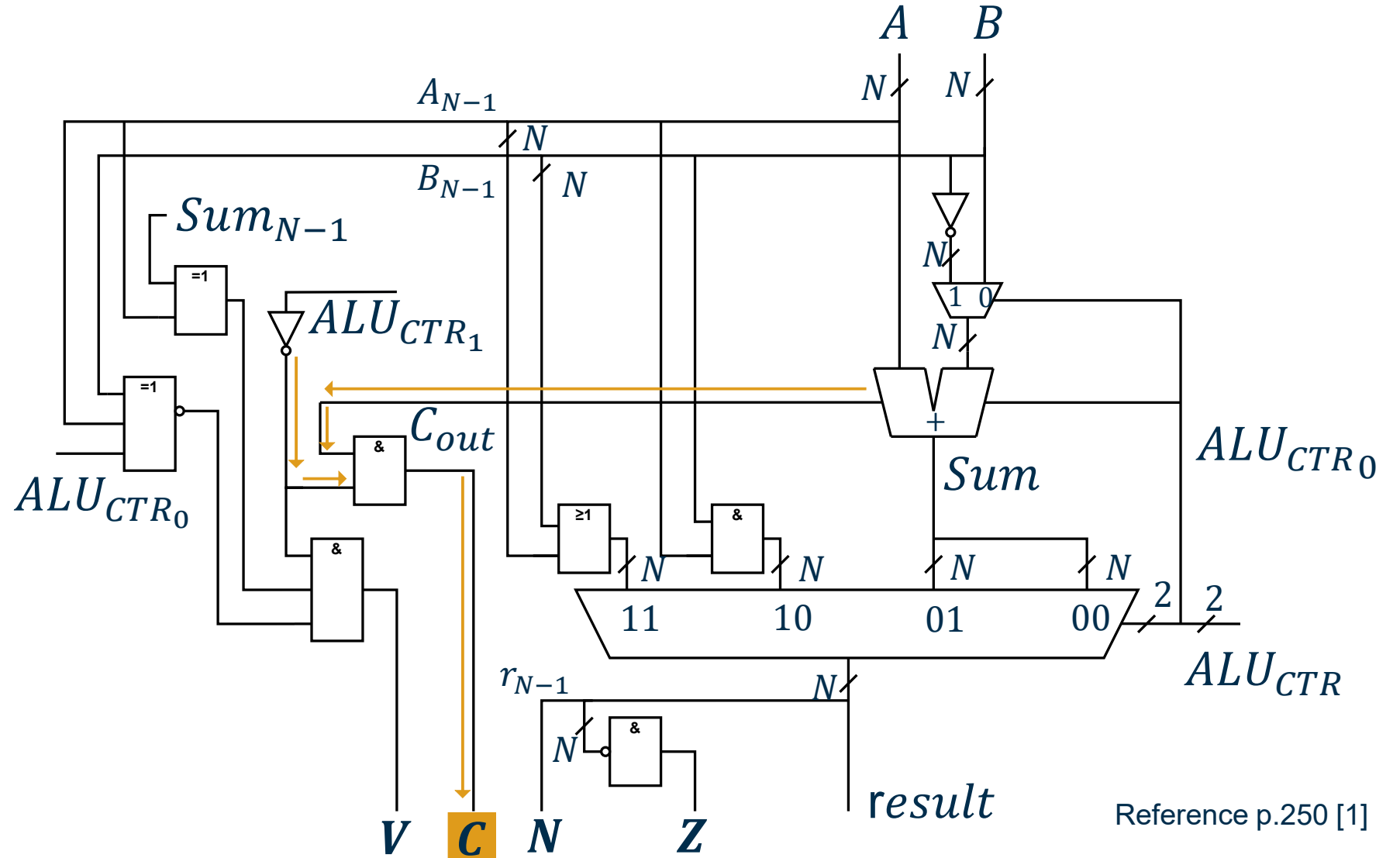


Reference p.250 [1]

Arithmetic/Logic Unit

Output Flags

- **C: Carry-Flag**
 - C_{out} ist gesetzt.
 - Der mode ist entweder ADD oder SUB. ($ALU_{CTR_1} = 0$)



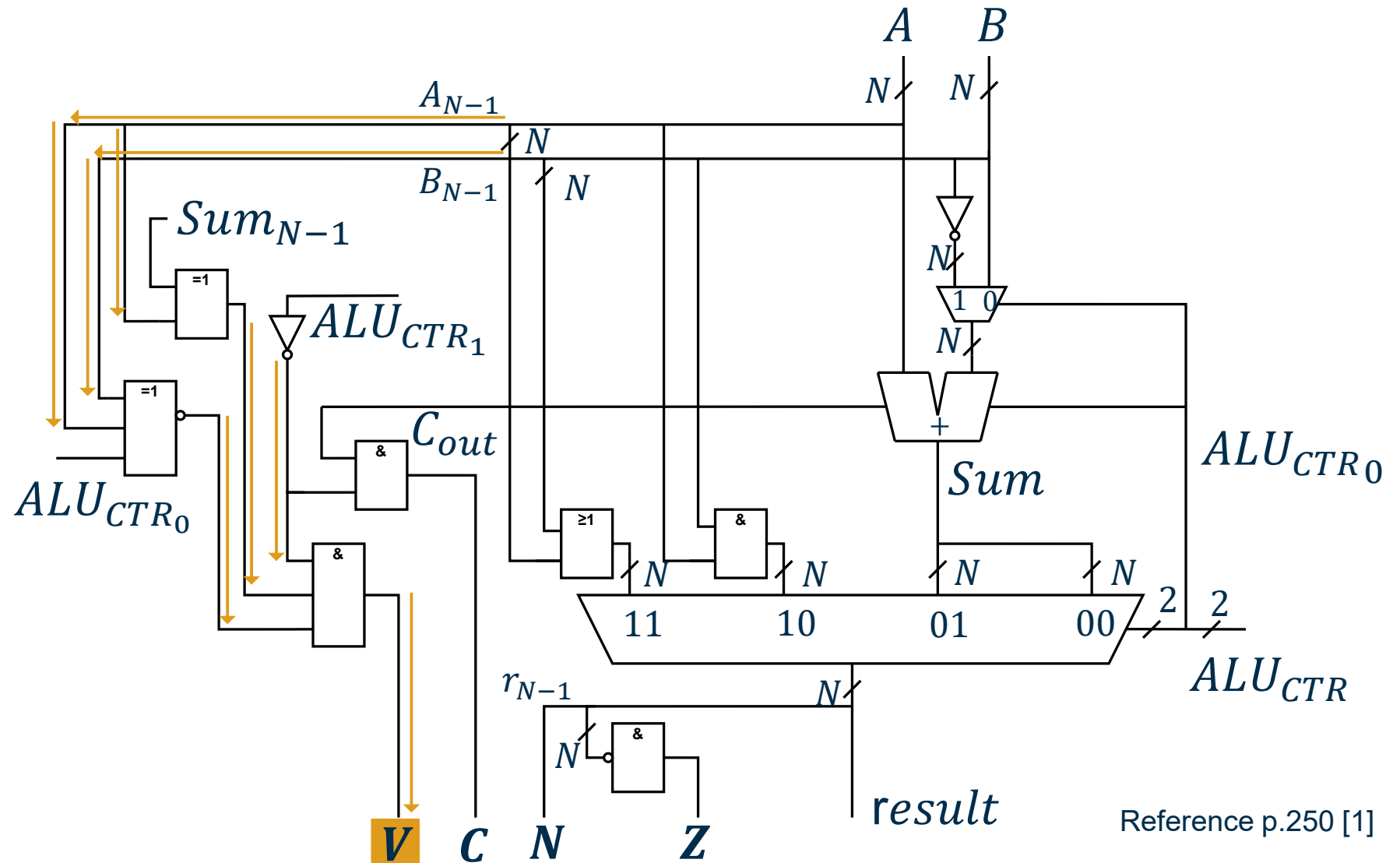
Reference p.250 [1]

Arithmetic/Logic Unit

Output Flags

■ V: Overflow-Flag

- Der mode ist entweder ADD oder SUB ($ALU_{CTR_1} = 0$)
- A hat ein anderes Vorzeichen als das Ergebnis des Adders. ($A_{N-1} \neq Sum_{N-1}$)
- A und B haben im mode ADD ($ALU_{CTR_0} = 0$) dasselbe Vorzeichen, oder im mode SUB ($ALU_{CTR_0} = 1$) unterschiedliche Vorzeichen.



Reference p.250 [1]



Shifter and Rotator

4

Shifter and Rotator

Einleitung

- Ein **Shifter** oder **Rotator** verschiebt Bits in einer Binärzahl nach links oder rechts. Es gibt häufig verwendete Shifter:

- Logical shifter



- Arithmetic shifter



- Rotator



Referenz S. 251 [1]

Shifter and Rotator

Logical Shifter

- Verschiebt die Bits nach links (**LSL**) oder rechts (**LSR**) und füllt die verbleibenden Bits mit Nullen auf.
- Die Operatoren <<,>> werden typischerweise verwendet.

Beispiel

11001 << 2 = 00100 (LSL 3)

11001 >> 2 = 00110 (LSR 3)

Referenz S. 251 [1]

Shifter and Rotator

Arithmetic shifter

- Bei Rechts Shifts (**ASR**) füllt der Shifter das MSB mit einer Kopie des vorherigen MSB. Bei Linksverschiebungen (**ASL**) füllt der Shifter die verbleibenden Bits mit Nullen (LSL). Dies wird für vorzeichenbehaftete Zahlen verwendet.
- Typischerweise werden die Operatoren <<<,>>> verwendet.

Beispiel

11001 <<< 2 = 00100 (ASL 3 / LSL 3)

11001 >>> 2 = 11110 (ASR 3)

Referenz S. 251 [1]

Shifter und Rotatoren

Rotator

- Bei Rechts Shifts (**ROR**) füllt der Rotator die verbleibenden Bits mit den von rechts verschobenen Bits. Bei Links Shifts (**ROL**) füllt der Rotator die verbleibenden Bits mit den von links verschobenen Bits.

Beispiel

11001 ROL 2 = 00111 (ROL 2)

11001 ROR 2 = 01110 (ROR 2)

Referenz S. 251 [1]

Shifter and Rotator

Multiplikation durch Shifts

Anmerkung

- Eine **Links Shift** ist ein Sonderfall der **Multiplikation**. Eine Linksverschiebung um N Bits multipliziert die Binärzahl mit 2^N .

Beispiel

$$\begin{array}{rcl} & 000011_2 & \Rightarrow \\ \ll 4 & & \Rightarrow \\ \hline & 110000_2 & \Rightarrow \end{array} \quad \begin{array}{rcl} & 3_{10} & \\ & 2^4 & \\ \hline & 48_{10} & \end{array}$$

Referenz S. 251 [1]

Shifter and Rotator

Division durch Shifts

Anmerkung

- Eine **arithmetischer Rechts Shift** ist ein Sonderfall der **Division**. Eine arithmetische Rechtsverschiebung um N Bits dividiert eine Binärzahl durch 2^N .

Beispiel

$$\begin{array}{rcl} & 11100_2 & \Rightarrow \\ & \quad 2 & \Rightarrow \\ \hline & 11111_2 & \Rightarrow \end{array} \quad \begin{array}{rcl} & -4_{10} & \\ & \quad 2^2 & \\ \hline & -1_{10} & \end{array}$$

$\ggg \quad /$

Referenz S. 251 [1]

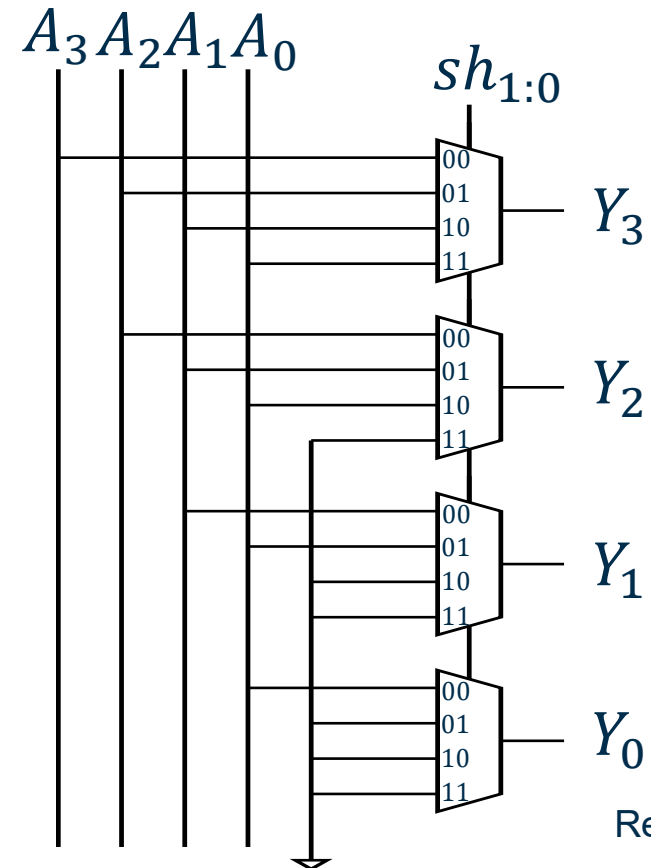
Shifter and Rotator

Implementierung

- Ein N-Bit Shifter kann mit N N:1-Multiplexern implementiert werden.

Beispiel Links Shift

$sh_{1:0}$	$Y_{3:0}$
00	$A_3 A_2 A_1 A_0$
01	$A_2 A_1 A_0 0$
10	$A_1 A_0 0 0$
11	$A_0 0 0 0$



Referenz S. 252 [1]

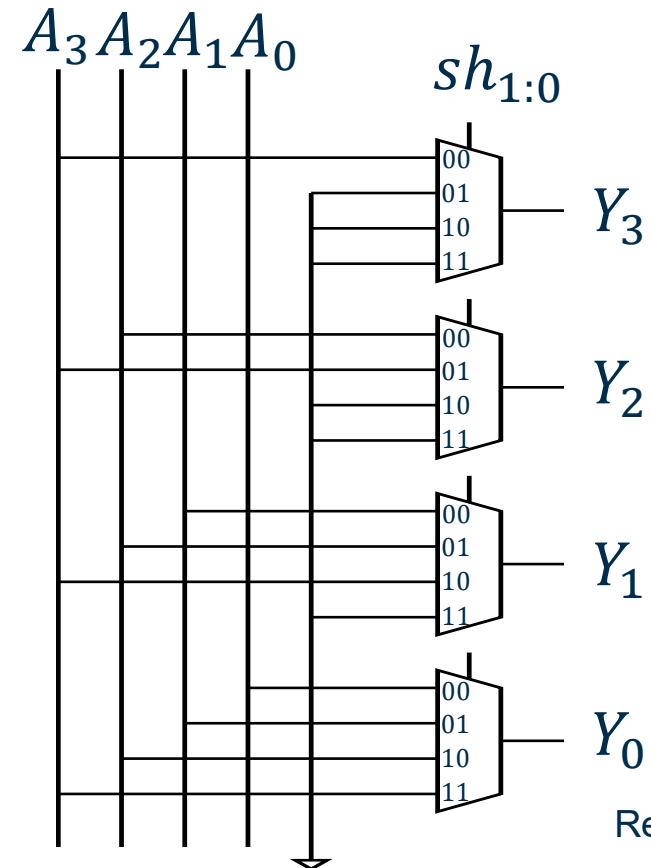
Shifter and Rotator

Implementierung

- Ein N-Bit Shifter kann mit N N:1-Multiplexern implementiert werden.

Beispiel Rechts Shift

$sh_{1:0}$	$Y_{3:0}$
00	$A_3 A_2 A_1 A_0$
01	0 $A_3 A_2 A_1$
10	0 0 $A_3 A_2$
11	0 0 0 A_3



Referenz S. 252 [1]



Multiplikation und Division

5

Multiplikation und Division

Division

- Wie bei Dezimalmultiplikation erfolgt die **Multiplikation** von unsigned Binärzahlen durch Summieren von **Teilprodukten**. Ein Teilprodukt ist das Produkt des Multiplikanden mit einer **einzelnen Ziffer des Multiplikators**.

Beispiel Vergleich zwischen dezimaler und binärer Multiplikation

Dezimalmultiplikation

$$\begin{array}{r} 230 \\ \times 142 \\ \hline 460 \\ 9200 \\ + 23000 \\ \hline 33660 \end{array}$$

$460 = 230 \times 2$
 $9200 = 230 \times 40$
 $23000 = 230 \times 100$
 $33660 = 230 \times 142$

Binäre Multiplikation

$$\begin{array}{r} 100 \\ \times 101 \\ \hline 100 \\ 0 \\ + 1000 \\ \hline 1100 \end{array}$$

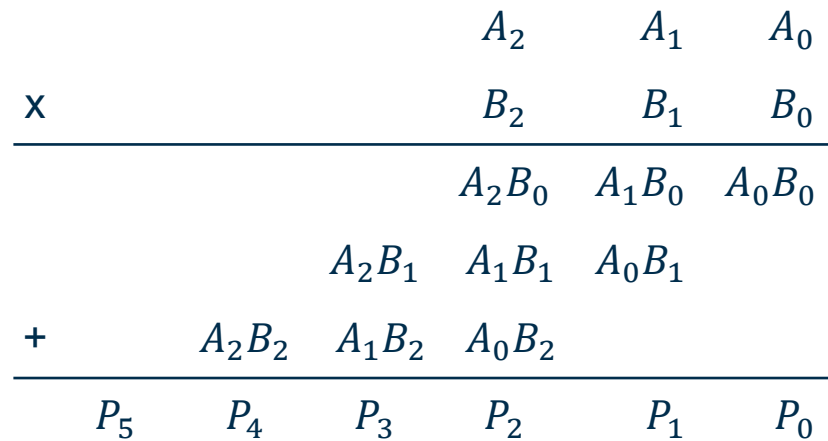
$100 \Rightarrow 8$
 $101 \Rightarrow 9$
 $100 = 0100 \times 1 \Rightarrow 8 \times 1$
 $0 = 0100 \times 00 \Rightarrow 8 \times 0$
 $1000 = 0100 \times 100 \Rightarrow 8 \times 8$
 $1100 \Rightarrow 72$

Siehe S. 253 [1]

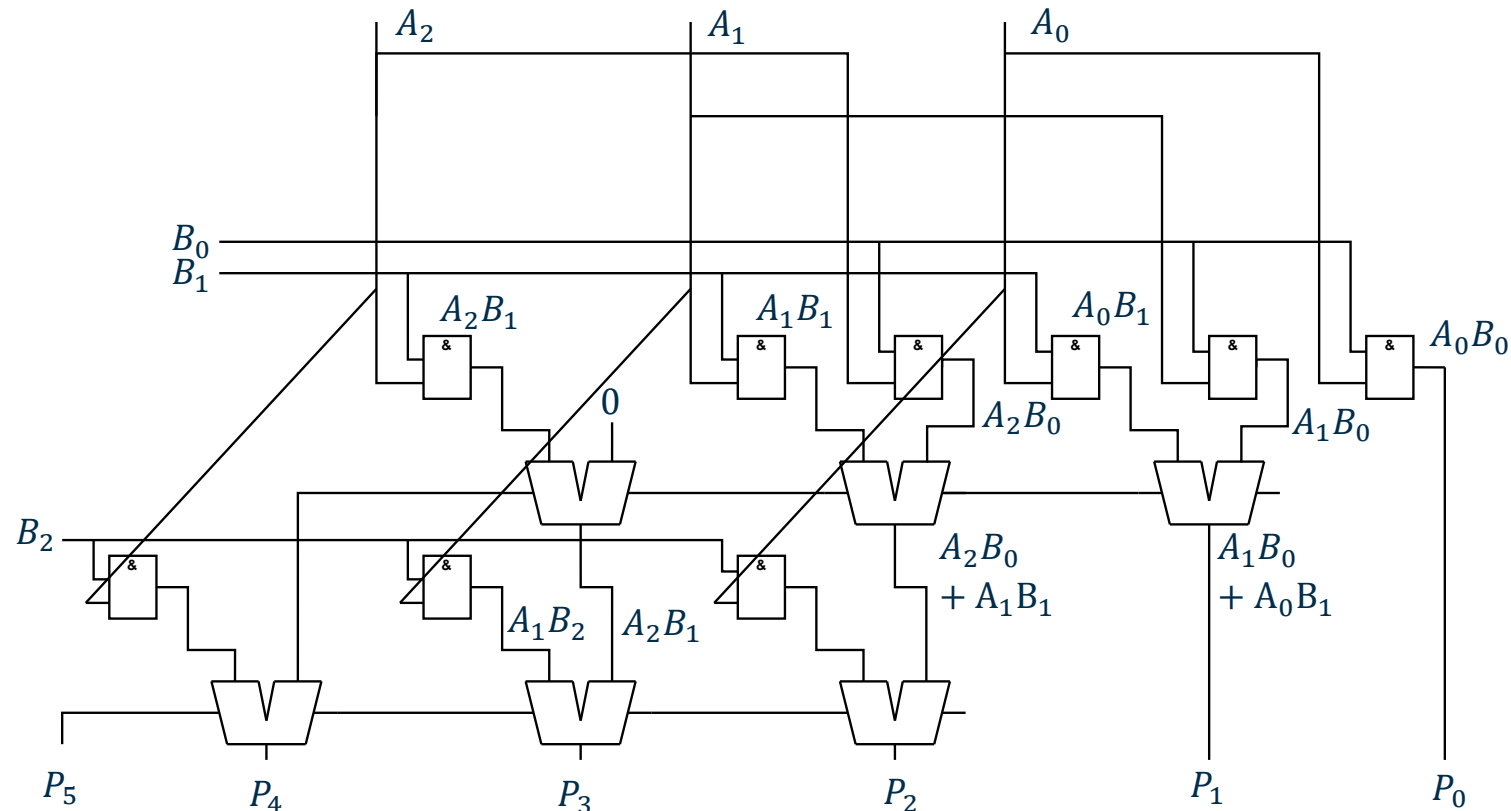
Multiplikation und Division

Multiplikator Implementation

- Jedes Teilprodukt ist das **um N Bit shifted Ergebnis** einer **UND-Operation** mit dem N-ten Multiplikator Bit und dem Multiplikanden.



Reference p.253 [1]



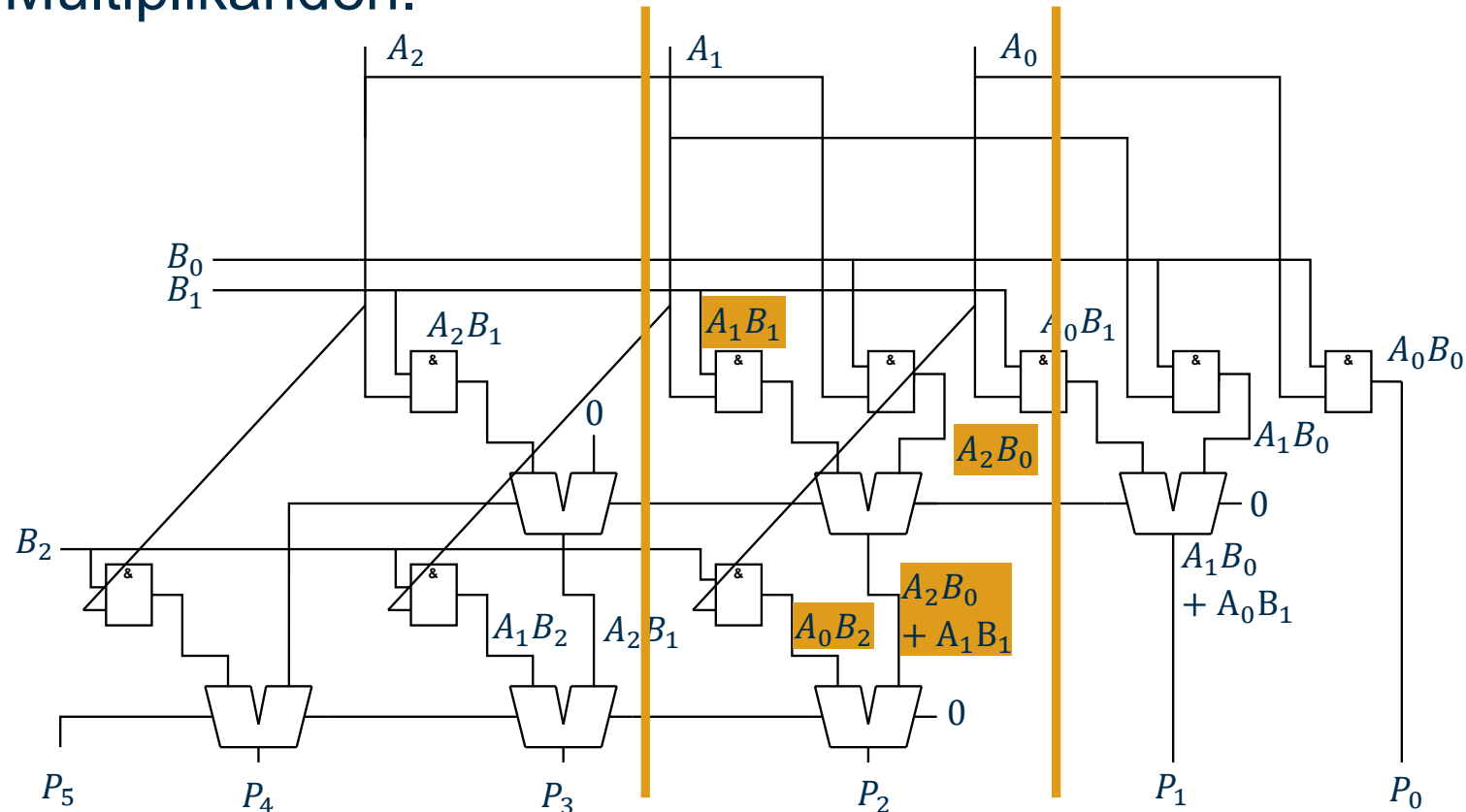
Multiplikation und Division

Multiplikator Implementation

- Jedes Teilprodukt ist das **um N Bit shifted Ergebnis** einer **UND-Operation** mit dem N-ten Multiplikator Bit und dem Multiplikanden.

			A_2	A_1	A_0	
x			B_2	B_1	B_0	
			A_2B_0	A_1B_0	A_0B_0	
			A_2B_1	A_1B_1	A_0B_1	
			A_2B_2	A_1B_2	A_0B_2	
+						
	P_5	P_4	P_3	P_2	P_1	P_0

Reference p.253 [1]



Multiplikation und Division

Divider

- Die binäre Division $A/B = Q + R/B$ kann mit dem folgenden Algorithmus für N-Bit-unsigned Zahlen im Bereich $[0, 2^{N-1}]$ berechnet werden:

$$R' = 0$$

for $i = N - 1$ *to* 0

$$R = \{ R' \ll 1, A_i \}$$

$$D = R - B$$

if $D < 0$ *then*

$$Q_i = 0, R' = R$$

else

$$Q_i = 1, R' = D$$

$$R = R'$$

Referenz S. 254 [1]

Multiplication und Division

Divider

Beispiel $16/3 = 5 R 1$

$$\begin{array}{r}
 10000 / 11 = xxxxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 0xxxx R 10000 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 00xxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 001xx R 00100 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 0010x R 00100 \\
 - 11 \\
 \hline
 1 \quad \Rightarrow 00101 R 00001
 \end{array}$$

$R' = 0$
for $i = N - 1$ *to* 0
 $R = \{ R' \ll 1, A_i \}$
 $D = R - B$
if $D < 0$ *then*
 $Q_i = 0, R' = R$
else
 $Q_i = 1, R' = D$
 $R = R'$

Q	R / R'	D
	00001 ₂	-2 ₁₀
0xxxx ₂	00001 ₂	

Multiplication und Division

Divider

Beispiel $16/3 = 5 R 1$

$$\begin{array}{r}
 10000 / 11 = xxxxx R 10000 \\
 - 11 \downarrow \\
 \hline
 10 \\
 - 11 \\
 \hline
 100 \\
 - 11 \\
 \hline
 10 \\
 - 11 \\
 \hline
 100 \\
 - 11 \\
 \hline
 1
 \end{array}
 \Rightarrow
 \begin{array}{l}
 0xxxx R 10000 \\
 00xxx R 10000 \\
 001xx R 00100 \\
 0010x R 00100 \\
 00101 R 00001
 \end{array}$$

$R' = 0$
for $i = N - 1$ *to* 0
 $R = \{ R' \ll 1, A_i \}$
 $D = R - B$
if $D < 0$ *then*
 $Q_i = 0, R' = R$
else
 $Q_i = 1, R' = D$
 $R = R'$

Q	R / R'	D
	00010 ₂	-1 ₁₀
00xxx ₂	00010 ₂	

Multiplication und Division

Divider

Beispiel $16/3 = 5 R 1$

$$\begin{array}{r}
 10000 / 11 = xxxxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 0xxxx R 10000 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 00xxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 001xx R 00100 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 0010x R 00100 \\
 - 11 \\
 \hline
 1 \quad \Rightarrow 00101 R 00001
 \end{array}$$

$R' = 0$
for $i = N - 1$ *to* 0
 $R = \{R' \ll 1, A_i\}$
 $D = R - B$
if $D < 0$ *then*
 $Q_i = 0, R' = R$
else
 $Q_i = 1, R' = D$
 $R = R'$

Q	R / R'	D
	00100 ₂	1 ₁₀
001xx ₂	00001 ₂	

Multiplication und Division

Divider

Beispiel $16/3 = 5 R 1$

$$\begin{array}{r}
 10000 / 11 = \text{xxxxx} R 10000 \\
 \underline{-11} \\
 10 \Rightarrow 0 \text{xxxx} R 10000 \\
 \underline{-11} \\
 100 \Rightarrow 00 \text{xxx} R 10000 \\
 \underline{-11} \\
 10 \Rightarrow 001 \text{xx} R 00100 \\
 \underline{-11} \\
 100 \Rightarrow 0010 \text{x} R 00100 \\
 \underline{-11} \\
 1 \Rightarrow 00101 R 00001
 \end{array}$$

$R' = 0$
for $i = N - 1$ *to* 0
 $R = \{ R' \ll 1, A_i \}$
 $D = R - B$
if $D < 0$ *then*
 $Q_i = 0, R' = R$
else
 $Q_i = 1, R' = D$
 $R = R'$

Q	R / R'	D
	00010 ₂	-1 ₁₀
0000x ₂	00010 ₂	

Multiplication und Division

Divider

Beispiel $16/3 = 5 R 1$

$$\begin{array}{r}
 10000 / 11 = xxxxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 0xxxx R 10000 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 00xxx R 10000 \\
 - 11 \\
 \hline
 10 \quad \Rightarrow 001xx R 00100 \\
 - 11 \\
 \hline
 100 \quad \Rightarrow 0010x R 00100 \\
 - 11 \\
 \hline
 1 \quad \Rightarrow 00101 R 00001
 \end{array}$$

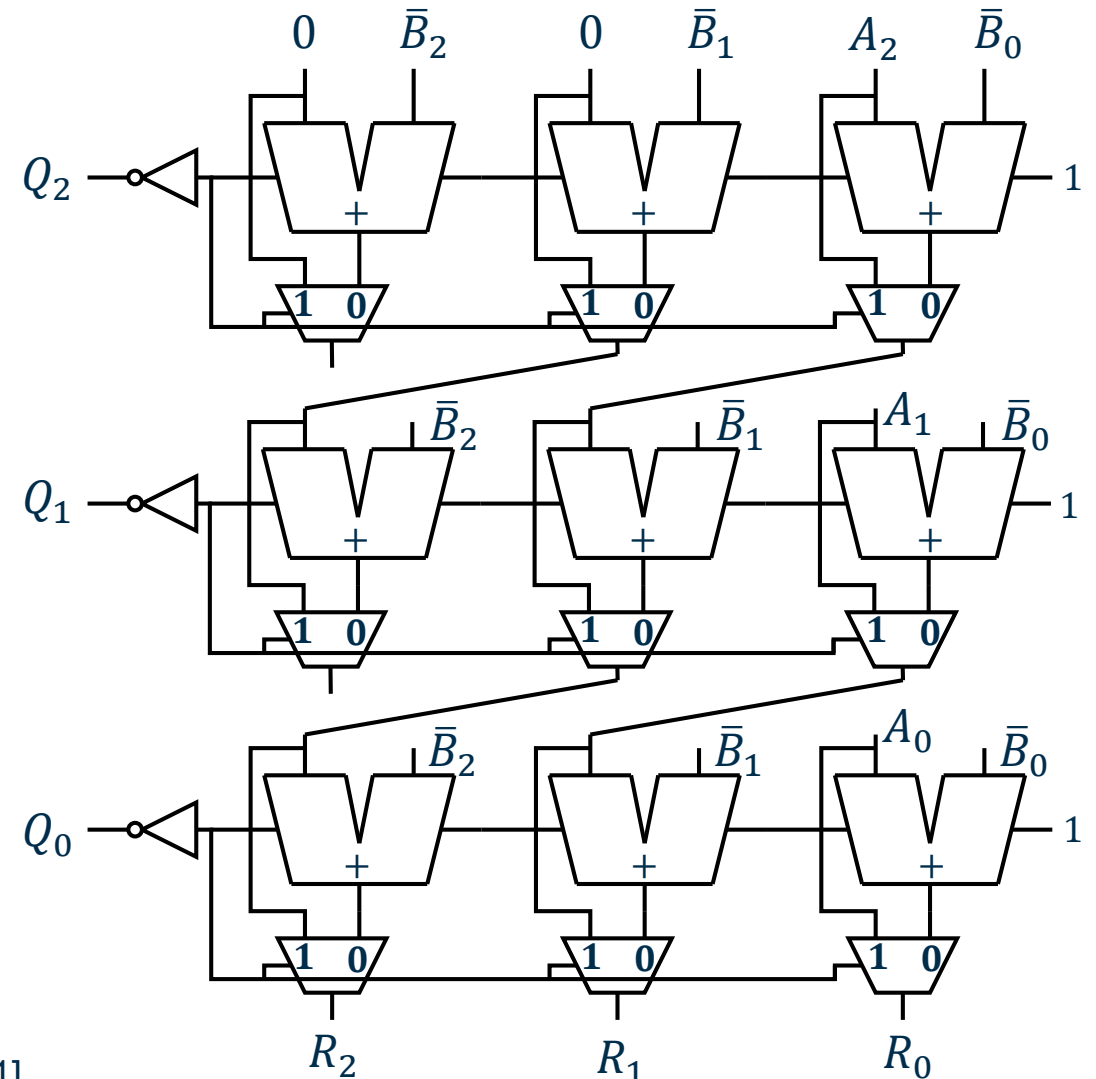
$R' = 0$
for $i = N - 1$ *to* 0
 $R = \{ R' \ll 1, A_i \}$
 $D = R - B$
if $D < 0$ *then*
 $Q_i = 0, R' = R$
else
 $Q_i = 1, R' = D$
 $R = R'$

Q	R / R'	D
	00100 ₂	1 ₁₀
00101 ₂	00001 ₂	
	00001 ₂	

Multiplikation und Division

Implementierung des Dividers

- Wie beim vorherigen Algorithmus muss der Divisor den Rest R in **Stufen** verarbeiten und an die nächsten Stufe übergeben.
- Der **Rest** muss entweder geschoben und mit dem nächsten Bit des Dividenden A aufgefüllt werden oder er muss durch den verbleibenden Wert **ersetzt**, dann geschoben und aufgefüllt werden, je nach Ergebnis der Subtraktion. (Entweder “passt” der Teiler in den Wert oder nicht)



S. 254 [1]



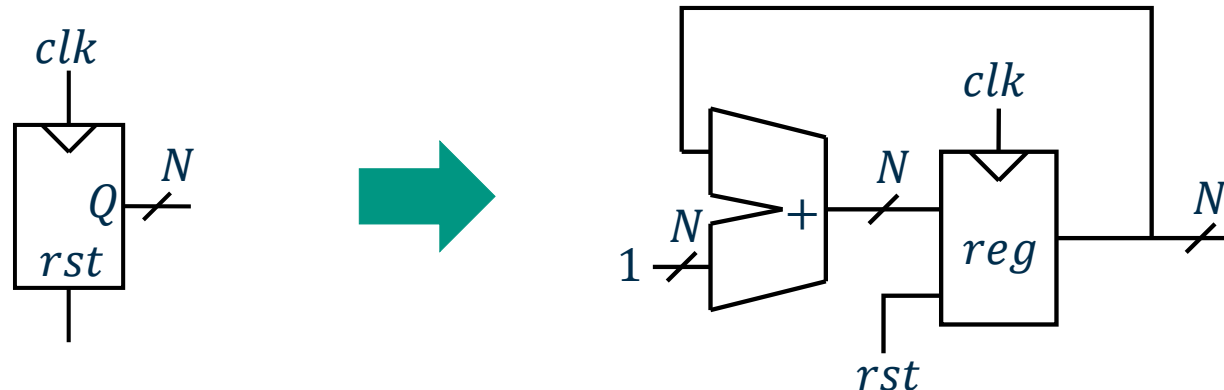
Sequentielle Bausteine

3

Sequentielle Bausteine

Counter

- Ein **N-bit binary counter** ist eine sequentielle arithmetische Schaltung mit:
 - Takt-Input
 - Reset Input
 - N-Bit Output Q
- Der **Reset** setzt den internen Zustand und den Output Q auf 0.
- Der Zähler zählt durch alle 2^N möglichen Ausgänge in **binärer Reihenfolge** und erhöht den Wert bei jeder **steigenden Taktflanke**.

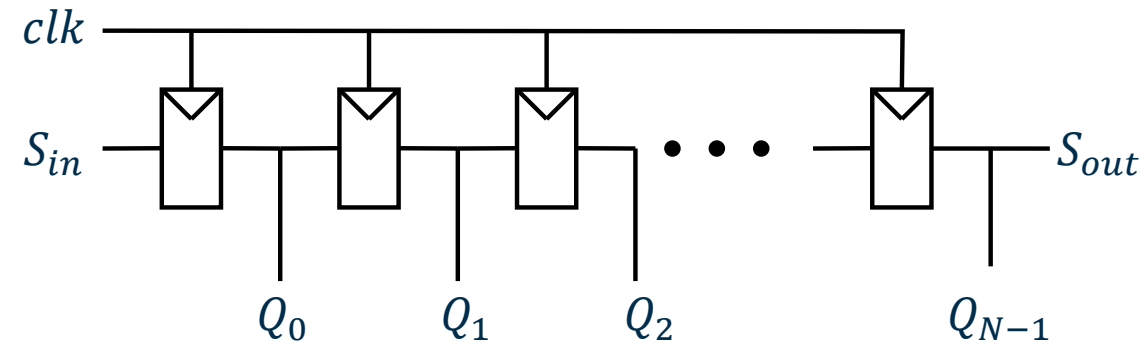


Referenz S. 260 [1]

Sequentielle Bausteine

Shift Register

- Ein **shift register** ist eine taktsignalgesteuerte **Kette** von N Registern, die in Reihe geschaltet sind.
- Ein shift register hat:
 - Takt-Input
 - Seriellen Input
 - Seriellen Output
 - N parallele Ausgänge
- Ein shift register kann als **Seriell-Parallel-Wandler** betrachtet werden. Nach N Zyklen sind die letzten N Eingänge parallel verfügbar unter Q .

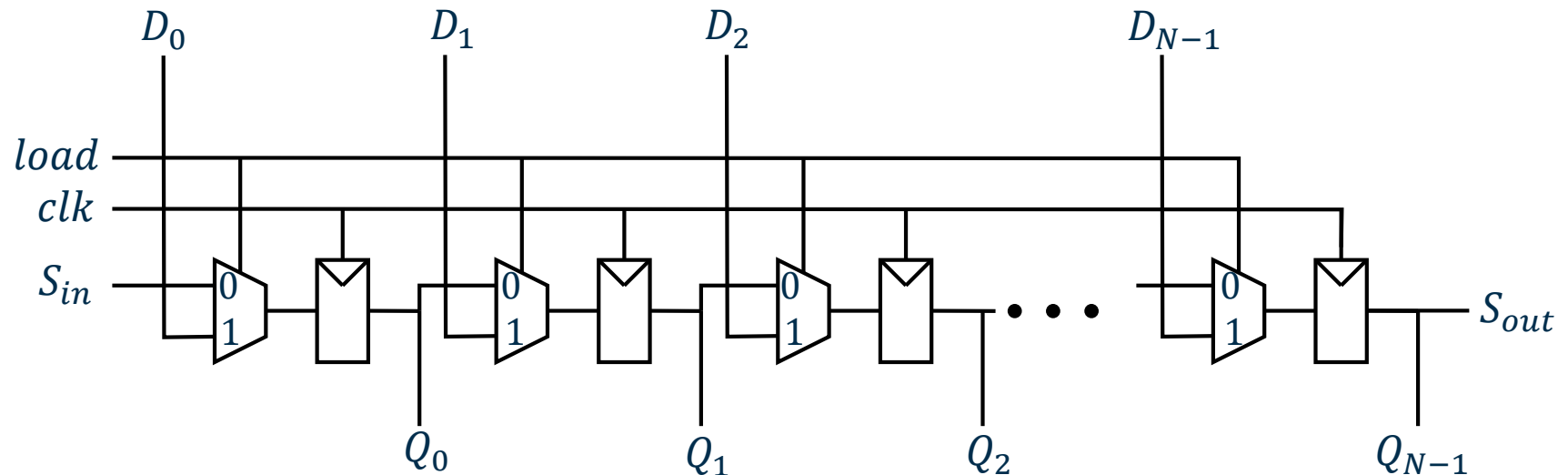


Referenz S. 261 [1]

Sequentielle Bausteine

Shift Register

- Das Schieberegister kann zu einem **Parallel-Seriell-Wandler** erweitert werden.
- Ein **load Signal** schaltet die Eingänge der Register um, um die parallelen Eingänge zu laden und sie dann normal zum Output zu shiften.



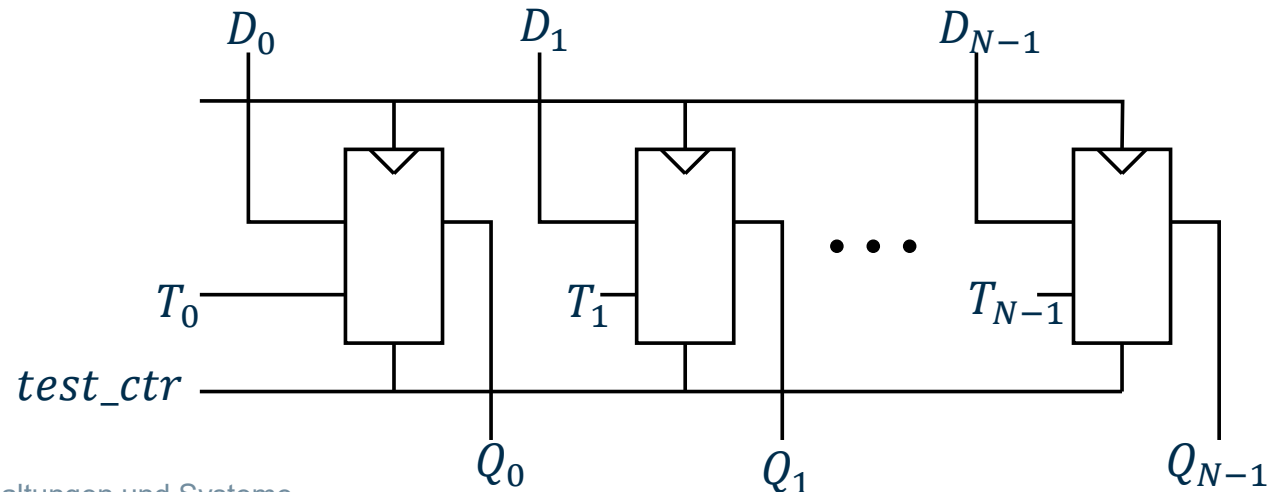
Referenz S. 263 [1]

Sequentielle Bausteine

Testing

- Testing von kombinatorischen Schaltungen ist unkompliziert, da ein Testvektor von Eingaben mit bekannten Ausgaben mit dem Ergebnis verglichen werden kann.
- Das Testen sequentieller Schaltungen ist aufgrund ihres internen Zustands schwieriger.
 - Beispielsweise erfordert die Überprüfung des Bitwechsels des Q_{31} Outputs eines 32-Bit-Zählers einen Reset und 2^{31} Taktimpulse.
- Daher ist es wünschenswert, den Wert aller internen Flipflops beobachten und steuern zu können.

- Lese alle Ausgänge Q .
- Schreibe Flipflops mit normalem oder T Input, gesteuert durch mode/select signal $test_ctr$.

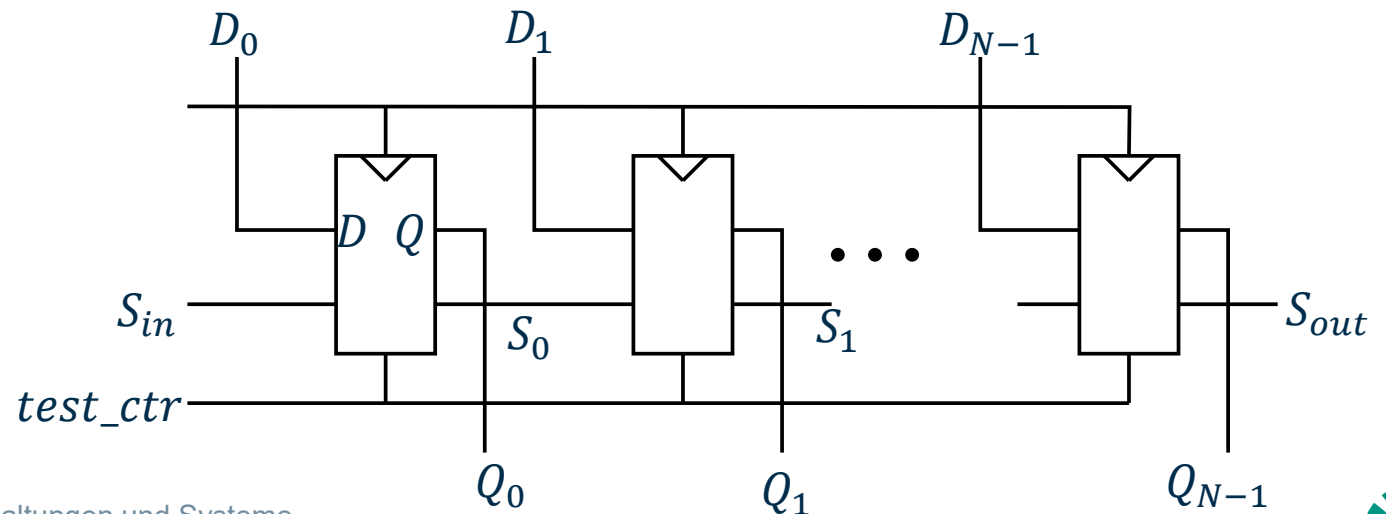


S. 263 [1]

Sequentielle Bausteine

Testing

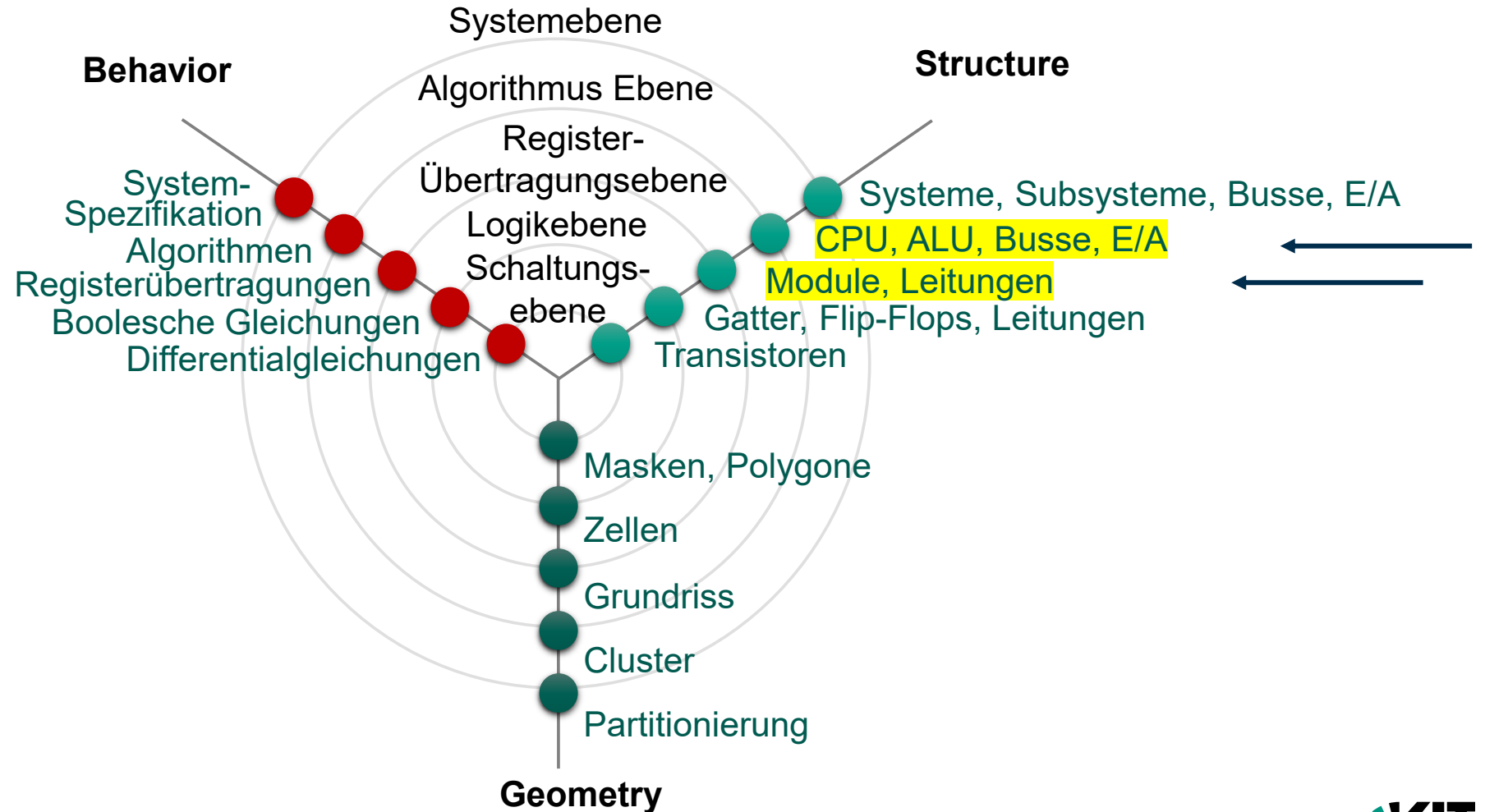
- Da die meisten Systeme über zu viele Flipflops verfügen, um einen dedizierten Schreib- und Leseinput anzuschließen, wird die Technik **scan chain** verwendet.
- Dafür werden alle Flipflops im System zu einem **Shift Register** verkettet.
- Im **Testmodus** verschieben die Flipflops ihren Inhalt seriell durch die Kette.
- Alle Flipflops sind dann über den **seriellen Input S_{in}** steuerbar und über den **seriellen Output S_{out}** beobachtbar.



S. 263 [1]

Advance Organizer

Y-Diagramm



Referenz

- [1] S. L. Harris und D. M. Harris, *Digitales Design und Computerarchitektur*, ARM®-Ausgabe. Waltham, MA: Morgan Kaufmann, 2016.

